
INTERNSHIP PROJECT REPORT

Multi-Camera Motion Capturing And Kinematic Analysis Application

Submitted by

Tuba Khan

AI & Robotics Intern

&

Kumar Abhishek

AI & Robotics Intern

Under the Supervision of

Dr. George Yuvaraj

Assistant Professor

NAMTECH

**School of Robotics & Automation Engineering
New Age Makers' Institute of Technology (NAMTECH)**

An Education Initiative of ArcelorMittal Nippon (AM/NS) Steel India

Gandhinagar, Gujarat – 382055

June 2026

NAM:TECH
We Inspire Humane Capital

Acknowledgements

We would like to express our sincere gratitude to our mentors at NAMTECH for their invaluable guidance, expert feedback, and continuous encouragement throughout every stage of this capstone project. The multi-camera working we did at NAMTECH, including the Logitech C920 webcams, Intel RealSense D435 depth camera, and the high-performance workstation was essential to enabling this research. Without these resources, the development of a functional multi-camera motion capture system would not have been possible within the internship timeline.

We are deeply thankful to the School of Robotics & Automation Engineering at New Age Makers' Institute of Technology (NAMTECH), and to ArcelorMittal Nippon (AM/NS) Steel India for their commitment to fostering a culture of hands-on technical innovation and applied research. The structure of the International Professional Master's Program challenged us to think rigorously about real-world engineering problems and produce solutions of industrial relevance.

We also extend our gratitude to the broader open-source computer vision community. The availability of OpenCV (Bradski, 2000), Google MediaPipe (Lugaresi et al., 2019), and the body of foundational research on stereo camera calibration, multi-object tracking, and visual SLAM including the landmark papers by Zhang (2000), Hartley and Zisserman (2003), Mildenhall et al. (2020), Dosovitskiy et al. (2021), and Campos et al. (2021) formed the academic and technical foundation upon which this project was built.

Finally, we thank our fellow interns in the AI and Robotics cohort at NAMTECH whose collaborative spirit and technical discussions kept this project motivated and progressing throughout.

Table of Contents

Chapter 1	Introduction	4
1.1	Background and Motivation	4-5
1.2	Problem Statement	5
1.3	Objectives of the Project	
1.4	Scope and Deliverables	5
Chapter 2	Literature Survey	6
2.1	Traditional Motion Capture Systems	6-7
2.2	Computer Vision and Deep Learning Approaches	...
2.3	LED-Based Marker Tracking	
2.4	Multi-Object Tracking with YOLO	
2.5	Neural Radiance Fields and 3D Scene Reconstruction	...
2.6	Vision Transformers in Computer Vision	13
2.7	Visual SLAM and Camera Calibration	13
Chapter 3	Methodology	16
3.1	System Architecture Overview	16-18
3.2	Hardware Setup and Camera Configuration	20-25
3.3	Project Folder Structure	25
3.4	Stereo Calibration Pipeline	25
3.5	LED Detection and 3D Tracking	
3.6	MediaPipe Skeleton Tracking	
Chapter 4	Results and Discussion	27-50
4.1	Stereo Calibration Results	51
4.2	LED Tracking XYZ Trajectory Results	53
4.3	MediaPipe Body Tracking Results	55
4.4	System Integration and Performance	57
Chapter 5	Challenges and Limitations	59
Chapter 6	Feasibility for Industrial Applications	60
Chapter 7	Conclusion and Future Work	61
	References	63

Chapter 1: Introduction

1.1 Background and Motivation

Motion capture (MoCap) is the process of digitally recording, in three-dimensional space, the movement of people or objects. It has become a critical enabling technology across a wide range of domains including feature film visual effects, video game character animation, clinical biomechanics, sports performance analysis, and increasingly, industrial ergonomics in manufacturing environments. Traditionally, MoCap systems have relied on expensive, proprietary hardware — typically arrays of infrared cameras costing \$50,000–\$500,000 per installation — combined with retro-reflective marker suits and specialised proprietary software. This cost barrier places professional MoCap technology out of reach for most academic institutions, small-scale research groups, and industrial facilities.

The convergence of three technological trends has made low-cost MoCap a realistic proposition in recent years: (1) the availability of high-quality consumer RGB cameras with stable USB interfaces; (2) the maturation of open-source computer vision libraries, particularly OpenCV, which provides production-grade stereo calibration and triangulation routines; and (3) the emergence of lightweight deep-learning-based pose estimation models such as Google MediaPipe (Lugaresi et al., 2019), which can detect 33 human body landmarks in real time on a standard CPU without GPU acceleration.

This project leverages all three trends to build a fully functional, low-cost motion capture system at NAMTECH that demonstrates professional-grade capabilities at approximately 1% of the cost of commercial alternatives.

1.2 Problem Statement

NAMTECH's Smart Manufacturing laboratory required a motion analysis capability to support research in industrial ergonomics — specifically, the ability to track and record human body movements and 3D point trajectories of tools and objects without acquiring commercial MoCap hardware. Existing solutions were either prohibitively expensive (Vicon, OptiTrack) or limited in 3D accuracy (single-camera markerless approaches). The specific engineering problem was: How can a multi-camera array of consumer-grade USB webcams, combined with open-source computer vision tools, provide accurate 3D motion reconstruction comparable to professional systems?

Two sub-problems were identified: (1) LED-based point tracking for precise 3D reconstruction of a single bright marker across calibrated camera pairs; and (2) markerless full-body skeleton tracking using a deep learning pose estimation model, allowing human motion to be captured without physically attaching markers to the subject.

1.3 Objectives of the Project

The primary objectives of this capstone project are:

1. To set up and synchronise a multi-camera array of 4 Logitech C920 RGB cameras and 1 Intel RealSense D435 depth camera for simultaneous video capture.
2. To implement a complete stereo calibration pipeline using OpenCV's checkerboard-based method, producing accurate rotation (R), translation (T), essential (E), and fundamental (F) matrices for all 6 camera pairs.
3. To implement a real-time LED detection and 3D triangulation pipeline that reconstructs the 3D world coordinates of a bright LED marker across multiple calibrated camera views.
4. To implement a real-time full-body pose estimation pipeline using Google MediaPipe Pose Landmarker, tracking 33 anatomical body landmarks per frame on a Logitech camera feed.
5. To log 3D XYZ coordinate data to Excel files and generate automated trajectory visualisation plots for subsequent analysis.

6. To evaluate the system against qualitative and quantitative accuracy benchmarks and document challenges, limitations, and future directions.

1.4 Scope and Deliverables

The scope of this project is bounded by the available hardware at NAMTECH (4 Logitech C920 cameras, 1 Intel RealSense D435, a Windows 11 laptop), the internship timeline, and the objective of building a reusable open-source Python codebase. The project does not aim to match the sub-millimetre accuracy of commercial infrared MoCap systems but rather to demonstrate that consumer-grade hardware can achieve practically useful 3D motion data for ergonomics and process monitoring applications.

Deliverables include: the full Python codebase (calibration, LED tracking, MediaPipe tracking), calibration data files (.npz), sample XYZ trajectory Excel logs, trajectory plot PNGs, and this written report.

Chapter 2: Literature Survey

2.1 Traditional Motion Capture Systems

Commercial optical MoCap systems such as Vicon and OptiTrack use arrays of high-frame-rate infrared cameras combined with retro-reflective markers placed on the subject's body. These systems achieve sub-millimetre positional accuracy and are the industry gold standard in film production, clinical biomechanics, and professional sports science. However, their cost (typically \$50,000–\$500,000 for a complete installation), requirement for controlled dark laboratory environments with calibrated marker placement, and need for specialised operator training make them unsuitable for most academic, educational, and small-scale industrial deployments.

Inertial measurement unit (IMU)-based systems such as Xsens MVN represent a portable alternative. They use networks of body-worn inertial sensors to estimate joint angles and body segment orientations through sensor fusion algorithms. While they eliminate the need for a fixed camera infrastructure, IMU-based systems suffer from sensor drift over time (accumulated integration error), require careful calibration for each subject, and are sensitive to electromagnetic interference — a significant issue in steel and heavy manufacturing environments. Electromagnetic MoCap systems achieve high accuracy in controlled indoor environments but are severely limited by interference from metallic structures, rendering them entirely unsuitable for factory floor deployment.

2.2 Computer Vision and Deep Learning Approaches to Motion Capture

The past decade has seen significant advances in computer-vision-based markerless motion capture driven by deep learning. OpenPose (Cao et al., 2017) introduced real-time multi-person 2D pose estimation using Part Affinity Fields and convolutional neural networks, enabling skeleton tracking without physical markers for the first time at scale. The MediaPipe framework (Lugaresi et al., 2019), developed at Google, extended this with the BlazePose architecture — a lightweight neural network that detects 33 anatomical body landmarks in real time with CPU-only inference, making it deployable without GPU hardware.

A parallel development of high relevance is the Vision Transformer (ViT), introduced by Dosovitskiy et al. (2021) in the landmark paper 'An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale'. ViT demonstrated that a pure Transformer architecture applied directly to sequences of image patches — without convolutional inductive biases — can match or exceed state-of-the-art CNN performance on image classification tasks when pre-trained on large datasets such as JFT-300M. The ViT architecture achieves 88.55% top-1 accuracy on ImageNet and requires substantially less compute than comparable ResNet models.

Neural Radiance Fields (NeRF), introduced by Mildenhall et al. (2020) in 'NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis', represent a breakthrough in implicit 3D scene representation. NeRF optimises a fully connected neural network to map continuous 5D coordinates (3D spatial position plus 2D viewing direction) to volume density and view-dependent colour, enabling photorealistic novel view synthesis from a sparse set of input images. While NeRF is not directly used in this project, its contribution to understanding multi-camera 3D reconstruction is highly relevant.

2.3 LED-Based Marker Tracking

LED-based point tracking provides a practical and computationally lightweight middle ground between expensive retro-reflective marker MoCap and fully markerless deep-learning approaches. A bright LED easily distinguishable from background in a camera frame can be reliably isolated using simple image processing: conversion to grayscale, brightness thresholding, and blob detection using connected component analysis or contour extraction. This approach is orders of magnitude more computationally efficient than neural-network-based detection, enabling real-time operation across multiple simultaneous camera feeds.

Prior work has validated LED marker approaches for metric accuracy. Studies in pendulum dynamics, robot end-effector tracking, and biomechanical measurement have demonstrated that LED-based systems can achieve centimetre-level 3D accuracy when cameras are properly calibrated using standard stereo calibration procedures.

2.4 Multi-Object Tracking with YOLO and DeepSORT

While this project focuses on single-point LED tracking and single-person MediaPipe tracking, the broader literature on multi-object tracking (MOT) provides important context and future directions. Kader et al. (2025), in 'Human Detection and Tracking with YOLO and SORT Tracking Algorithm', demonstrate a complete tracking-by-detection (TBD) pipeline combining YOLOv8 and YOLO11 object detection models with SORT and DeepSORT trackers for real-time human tracking in indoor and outdoor surveillance scenarios.

Their work shows that optimised YOLO11m with DeepSORT achieves 74% Multiple Object Tracking Accuracy (MOTA) on a custom recorded dataset of 8,427 frames across 7 video sequences. This finding underscores the importance of detection quality for tracking performance, a principle that directly applies to LED detection robustness in the present system.

2.5 Neural Radiance Fields and 3D Scene Reconstruction

Mildenhall et al. (2020) present NeRF as a method that achieves state-of-the-art results for novel view synthesis by optimising an underlying continuous 5D neural radiance field using a sparse set of input images. Key technical contributions include positional encoding of input coordinates to enable high-frequency scene representation, and a hierarchical sampling strategy that allocates network capacity to visible scene content.

The relevance to this project is twofold. First, NeRF demonstrates the critical importance of accurate camera calibration (intrinsics and extrinsics) for any 3D reconstruction task — the same calibration data this project computes using OpenCV stereo calibration. Second, NeRF's use of multi-view geometry and triangulation principles provides theoretical grounding for the stereo triangulation approach used in LED 3D reconstruction.

2.6 Vision Transformers in Computer Vision

The Vision Transformer (ViT) paper by Dosovitskiy et al. (2021) has been foundational in shifting the computer vision community's understanding of inductive biases and scalability. By demonstrating that pure Transformer architectures — without convolution — can match ResNet performance on image classification when pre-trained at sufficient scale, ViT opened a new design space for visual understanding models. In the context of this project, ViT's influence is most visible in the MediaPipe BlazePose architecture, which leverages attention mechanisms for robust landmark detection across diverse body poses and occlusion conditions.

2.7 Visual SLAM and Camera Calibration

ORB-SLAM3, presented by Campos et al. (2021), represents the most comprehensive open-source SLAM library available, supporting monocular, stereo, and RGB-D cameras with optional IMU integration. It demonstrates stereo-inertial SLAM achieving average accuracy of 3.5 cm on the EuRoC drone dataset and 9 mm on the TUM-VI benchmark under rapid hand-held motion — performance levels directly relevant to the motion capture use case of this project.

The stereo calibration pipeline used in this project draws directly from the computer vision foundations underlying ORB-SLAM3: Zhang's (2000) checkerboard method for intrinsic calibration, and stereo calibration using the `CALIB_FIX_INTRINSIC` flag in OpenCV to compute accurate extrinsic rotation (R) and translation (T) matrices between camera pairs.

Chapter 3: Methodology

3.1 System Architecture Overview

The system is architected as a multi-threaded Python application operating on a Windows 11 host. At its highest level, the system consists of three parallel subsystems: (1) a multi-camera video acquisition layer that opens all cameras simultaneously using separate CameraThread instances; (2) a stereo calibration module that computes and stores geometric relationships between cameras offline; and (3) two parallel online tracking modules — LED brightness-threshold detection with stereo triangulation, and MediaPipe deep-learning body landmark detection.

The choice of Python as the implementation language is motivated by the availability of mature, well-documented bindings for OpenCV (cv2), MediaPipe, Intel RealSense (pyrealsense2), NumPy, openpyxl, and matplotlib. The multi-threaded design prevents any single camera's frame acquisition or detection computation from blocking the others, ensuring near-real-time operation across all five camera feeds simultaneously.

3.2 Hardware Setup and Camera Configuration

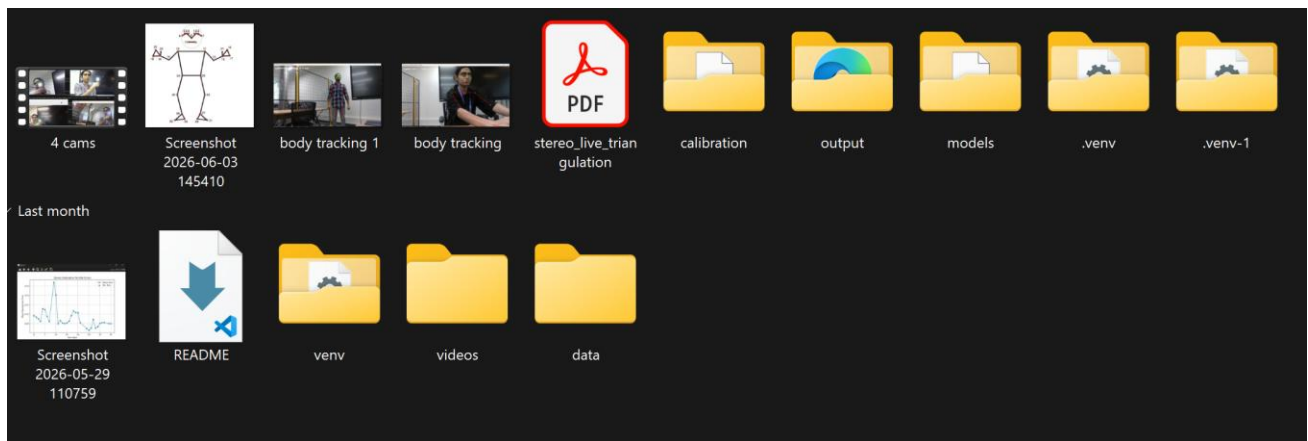
The complete hardware bill of materials for this system is:

- 4 x Logitech C920 HD Pro Webcams (1080p capable, operated at 640x480 for USB bandwidth management, 30 fps) connected via USB 3.0 to host laptop ports.
- 1 x Intel RealSense D435 Depth Camera (RGB + structured light depth, 30 fps) accessed via the pyrealsense2 SDK independently of the DirectShow camera index system.
- 1 x Host Laptop: Windows 11, Intel Core i7, 16 GB RAM, USB 3.0 ports. The laptop's internal webcam at USB index 0 is excluded from the capture pipeline to preserve USB bandwidth.
- 1 x Checkerboard calibration target: printed on rigid A3 card, 8x6 squares with inner corner array of 7x5 corners, 28 mm square side length.
- 1 x LED light source: bright white LED on a short cable, used for point-tracking experiments. The LED was held by hand and mounted on a pendulum for trajectory capture sessions.

Camera resolution was reduced from 1280x720 to 640x480 after initial testing revealed that four simultaneous 720p streams saturated the USB 3.0 host controller, causing intermittent camera dropout. At 640x480, all four cameras initialise and stream reliably. Camera indices [0,1,2,3] were confirmed using `find_camera.py` at the start of each session.

3.3 Project Folder Structure

The project directory is organised into the following structure to separate calibration data, model files, raw outputs, and processing scripts:



- motion_capture_project/scripts/ — All Python processing scripts: find_camera.py, quad_stereo_pipeline.py, led_tracking.py, mediapipe_tracking.py
- motion_capture_project/calibration_images/ — Synchronised checkerboard frames in subfolders cam0/ through cam3/
- motion_capture_project/calibration/ — Output .npz stereo calibration files for all 6 camera pairs
- motion_capture_project/outputs/ — XYZ trajectory Excel logs (.xlsx) and trajectory plot PNGs (xyz_plot_*.png)
- motion_capture_project/models/ — MediaPipe Pose Landmarker model file (pose_landmarker_full.task)
- motion_capture_project/stereo_images/ — Organised calibration image sets per camera
- motion_capture_project/data/ — Raw video recordings (.avi) from optional session recording

ANNEXURE A – Camera Clibartion.py

```
import argparse
import os
import threading
import time
import urllib.request
from datetime import datetime
```

```
import cv2
import numpy as np
from openpyxl import Workbook
```

```
# —— MediaPipe lazy import
```

```
try:
```

```
    import mediapipe as mp
    from mediapipe.tasks import python as mp_python
    from mediapipe.tasks.python import vision as mp_vision
    _MP_AVAILABLE = True
```

```
except ImportError:
```

```
    _MP_AVAILABLE = False
    print("[WARN] mediapipe not installed — cam0 will show plain video feed.")
```

```
#
```

```

# CONSTANTS
#


---




---


DISPLAY_W    = 480
DISPLAY_H    = 270
LED_CAM_INDICES = [1, 2, 3, 5] # 4 LED cams
POSE_CAM_INDEX = 0             # MediaPipe cam

MODEL_URL = ("https://storage.googleapis.com/mediapipe-models/"
             "pose_landmarker/pose_landmarker_full/float16/latest/"
             "pose_landmarker_full.task")
MODEL_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)),
                           "models", "pose_landmarker_full.task")

POSE_CONNECTIONS = [
    (0,1),(1,2),(2,3),(3,7),(0,4),(4,5),(5,6),(6,8),(9,10),
    (11,12),(11,13),(13,15),(15,17),(15,19),(15,21),(17,19),
    (12,14),(14,16),(16,18),(16,20),(16,22),(18,20),
    (11,23),(12,24),(23,24),
    (23,25),(25,27),(27,29),(27,31),(29,31),
    (24,26),(26,28),(28,30),(28,32),(30,32),
]

# Per-camera LED colors
LED_COLORS = [
    (0, 255, 0), # LED1 – green (stereo A)
    (0, 165, 255), # LED2 – orange (stereo B)
    (0, 255, 255), # LED3 – cyan (extra)
    (255, 0, 255), # LED4 – magenta (extra)
]

_frame_counter = 0 # for pulsing animation

#


---




---


# CAMERA THREAD
#


---




---


class CameraThread:
    def __init__(self, cam_id, index, width=1280, height=720):
        self.cam_id = cam_id
        self.index = index
        self.width = width
        self.height = height
        self.frame = None
        self.ok = False
        self.error = None
        self.backend = "?"
        self._lock = threading.Lock()
        self._stop = threading.Event()
        self._thread = threading.Thread(target=self._run, daemon=True)

    def start(self): self._thread.start()

```

```

def stop(self):
    self._stop.set()
    self._thread.join(timeout=3.0)

def get_frame(self):
    with self._lock:
        return self.frame.copy() if self.frame is not None else None

def _open(self):
    for api, bname in [(cv2.CAP_DSHOW, "dshow"),
                      (cv2.CAP_MSMF, "msmf"),
                      (cv2.CAP_ANY, "any")]:
        for _ in range(3):
            cap = cv2.VideoCapture(self.index, api)
            if not cap.isOpened():
                cap.release(); time.sleep(0.4); continue
            cap.set(cv2.CAP_PROP_FRAME_WIDTH, self.width)
            cap.set(cv2.CAP_PROP_FRAME_HEIGHT, self.height)
            cap.set(cv2.CAP_PROP_FPS, 30)
            for _ in range(60):
                ret, fr = cap.read()
                if ret and fr is not None and fr.size > 0 and float(np.mean(fr)) > 1.0:
                    self.backend = bname
                    return cap
            cap.release(); time.sleep(0.5)
    return None

def _run(self):
    cap = None
    for attempt in range(4):
        cap = self._open()
        if cap is not None:
            break
        print(f" cam{self.cam_id} (idx={self.index}): attempt {attempt+1} failed...")
        time.sleep(1.0)
    if cap is None:
        self.error = f"cam{self.cam_id} (idx={self.index}): failed all attempts"
        return
    self.ok = True
    print(f" cam{self.cam_id} READY idx={self.index} ({self.backend})")
    while not self._stop.is_set():
        ret, frame = cap.read()
        if ret and frame is not None and frame.size > 0:
            with self._lock:
                self.frame = frame
        else:
            time.sleep(0.005)
    cap.release()

```

#

SMART LED DETECTION (ported from led tracking.py + improved)

#

```

def _score_contours(contours, gray, frame_area, min_area):

```

```

"""Score all contours, return best (center, contour) or (None, None)."""
h, w = gray.shape[:2]
best_score = -1.0
best_cnt = None

for c in contours:
    area = cv2.contourArea(c)
    if area < min_area:
        continue
    if area > 0.08 * frame_area:      # reject huge bright regions (>8% of frame)
        continue
    bx, by, bw2, bh2 = cv2.boundingRect(c)
    aspect = max(bw2, bh2) / max(min(bw2, bh2), 1)
    if aspect > 3.5:                  # reject long thin blobs (fluorescent tubes)
        continue

    blob_mask = np.zeros((h, w), np.uint8)
    cv2.drawContours(blob_mask, [c], -1, 255, -1)
    mean_bright = cv2.mean(gray, mask=blob_mask)[0]

    perimeter = cv2.arcLength(c, True)
    circularity = 4.0 * np.pi * area / (perimeter ** 2 + 1e-6)

    # Score = brightness × circularity / aspect_penalty
    score = mean_bright * circularity / aspect
    if score > best_score:
        best_score = score
        best_cnt = c

if best_cnt is None:
    return None, None

m = cv2.moments(best_cnt)
if m["m00"] == 0:
    return None, None
return (float(m["m10"] / m["m00"]), float(m["m01"] / m["m00"])), best_cnt

def detect_led(frame, thresh, min_area=20):
    """
    Smart LED detection:
    1. Find brightest pixel (minMaxLoc)
    2. Threshold tightly around that brightness
    3. Score blobs by brightness × circularity / aspect
    4. Fallback: retry at thresh-20 if nothing found

    Returns (center_xy_or_None, debug_binary_image)
    """
    if frame is None or frame.size == 0:
        return None, np.zeros((10, 10), np.uint8)

    h, w = frame.shape[:2]
    frame_area = h * w
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)

    k3 = np.ones((3, 3), np.uint8)
    k11 = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (11, 11))

```

```
_ , max_val, _, _ = cv2.minMaxLoc(gray)
```

```
def _detect_at(t):
```

```
    _ , mask = cv2.threshold(gray, max(t, max_val - 15), 255, cv2.THRESH_BINARY)
    mask = cv2.morphologyEx(mask, cv2.MORPH_OPEN, k3)
    merged = cv2.dilate(mask, k11, iterations=2)
    cnts, _ = cv2.findContours(merged, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    center, cnt = _score_contours(cnts, gray, frame_area, min_area)
    return center, cnt, merged
```

```
# — Primary attempt
```

```
if max_val >= thresh:
    center, cnt, debug = _detect_at(thresh)
    if center is not None:
        return center, debug
```

```
# — Fallback: lower threshold by 20
```

```
lower = max(thresh - 20, 80)
center, cnt, debug = _detect_at(lower)
return center, debug
```

```
#
```

```
# DRAW HELPERS
```

```
#
```

```
def draw_led(frame, pt, color):
```

```
    """Large pulsing marker: outer ring + inner ring + crosshair + centre dot."""
```

```
    global _frame_counter
```

```
    if pt is None:
```

```
        return
```

```
    cx, cy = int(round(pt[0])), int(round(pt[1]))
```

```
    pulse = int(6 * np.sin(_frame_counter * 0.2))
```

```
    R_outer = 36 + pulse
```

```
    R_inner = 16
```

```
    LINE_LEN = 55
```

```
    WHITE = (255, 255, 255)
```

```
    cv2.circle(frame, (cx, cy), R_outer + 5, WHITE, 2) # glow halo
```

```
    cv2.circle(frame, (cx, cy), R_outer, color, 3) # outer ring
```

```
    cv2.circle(frame, (cx, cy), R_inner, color, 2) # inner ring
```

```
    cv2.circle(frame, (cx, cy), 6, color, -1) # centre dot
```

```
    cv2.circle(frame, (cx, cy), 6, WHITE, 1)
```

```
    gap = R_inner + 2
```

```
    cv2.line(frame, (cx - LINE_LEN, cy), (cx - gap, cy), color, 2)
```

```
    cv2.line(frame, (cx + gap, cy), (cx + LINE_LEN, cy), color, 2)
```

```
    cv2.line(frame, (cx, cy - LINE_LEN), (cx, cy - gap), color, 2)
```

```
    cv2.line(frame, (cx, cy + gap), (cx, cy + LINE_LEN), color, 2)
```

```
    _frame_counter += 1
```

```
#
```

```
# CALIBRATION + TRIANGULATION
```

```
#
```

```
def load_calibration(path):
    if not os.path.exists(path):
        raise FileNotFoundError(f"Calibration file not found: {path}")
    d = np.load(path)
    K1 = d["cameraMatrix1"] if "cameraMatrix1" in d else d["K1"]
    D1 = d["distCoeffs1"] if "distCoeffs1" in d else d["D1"]
    K2 = d["cameraMatrix2"] if "cameraMatrix2" in d else d["K2"]
    D2 = d["distCoeffs2"] if "distCoeffs2" in d else d["D2"]
    return K1, D1, K2, D2, d["R"], d["T"]

def resolve(arg):
    if os.path.isabs(arg): return arg
    p = os.path.abspath(arg)
    if os.path.exists(p): return p
    root = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
    return os.path.abspath(os.path.join(root, arg))

def triangulate(pt1, pt2, K1, D1, K2, D2, R, T):
    P1 = np.hstack((np.eye(3), np.zeros((3, 1))))
    P2 = np.hstack((R, T))
    n1 = cv2.undistortPoints(np.array([[[pt1[0], pt1[1]]]], np.float64), K1, D1)
    n2 = cv2.undistortPoints(np.array([[[pt2[0], pt2[1]]]], np.float64), K2, D2)
    Xh = cv2.triangulatePoints(P1, P2, n1.reshape(2, 1), n2.reshape(2, 1))
    w = float(Xh[3, 0])
    return None if abs(w) < 1e-9 else (Xh[:3, 0] / w).astype(np.float64)
```

```
#
```

```
# MEDIAPIPE HELPERS
```

```
#
```

```
def ensure_model():
    os.makedirs(os.path.dirname(MODEL_PATH), exist_ok=True)
    if not os.path.exists(MODEL_PATH):
        print(f"Downloading pose model → {MODEL_PATH}")
        try:
            urllib.request.urlretrieve(MODEL_URL, MODEL_PATH)
            print(" Download complete.")
        except Exception as e:
            print(f" [WARN] Download failed: {e}")
            return False
    return True

def build_landmarker():
    if not _MP_AVAILABLE or not ensure_model():
        return None
    try:
        opts = mp_vision.PoseLandmarkerOptions(
```

```

        base_options=mp_python.BaseOptions(model_asset_path=MODEL_PATH),
        running_mode=mp_vision.RunningMode.VIDEO,
        num_poses=1,
        min_pose_detection_confidence=0.5,
        min_pose_presence_confidence=0.5,
        min_tracking_confidence=0.5,
        output_segmentation_masks=False,
    )
    return mp_vision.PoseLandmarker.create_from_options(opts)
except Exception as e:
    print(f" [WARN] MediaPipe init failed: {e}")
    return None

def draw_skeleton(frame, landmarks, h, w):
    pts = {i: (int(lm.x * w), int(lm.y * h)) for i, lm in enumerate(landmarks)}
    for a, b in POSE_CONNECTIONS:
        if (landmarks[a].visibility or 0) > 0.3 and (landmarks[b].visibility or 0) > 0.3:
            cv2.line(frame, pts[a], pts[b], (0, 0, 200), 2, cv2.LINE_AA)
    for i, lm in enumerate(landmarks):
        if (lm.visibility or 0) > 0.3:
            cv2.circle(frame, pts[i], 6, (255, 255, 255), -1)
            cv2.circle(frame, pts[i], 5, (0, 220, 0), -1, cv2.LINE_AA)

#
=====
#
#
=====
#
=====

class Logger:
    def __init__(self, out_dir):
        os.makedirs(out_dir, exist_ok=True)
        ts = datetime.now().strftime("%Y%m%d_%H%M%S")
        self.excel = os.path.join(out_dir, f"5cam_log_{ts}.xlsx")
        self.plot = os.path.join(out_dir, f"5cam_plot_{ts}.png")
        self.rows = []
        self._last = 0

    def add(self, idx, led_pts, xyz_raw, xyz_sm):
        def px(pt, i): return round(float(pt[i]), 1) if pt is not None else None
        def xv(arr, i):
            if arr is None: return None
            try: return round(float(arr[i]), 4)
            except: return None

        self.rows.append({
            "sample": idx,
            "ts": datetime.now().isoformat(timespec="milliseconds"),
            "led1_x": px(led_pts[0], 0), "led1_y": px(led_pts[0], 1),
            "led2_x": px(led_pts[1], 0), "led2_y": px(led_pts[1], 1),
            "led3_x": px(led_pts[2], 0), "led3_y": px(led_pts[2], 1),
            "led4_x": px(led_pts[3], 0), "led4_y": px(led_pts[3], 1),
            "x_raw": xv(xyz_raw, 0), "y_raw": xv(xyz_raw, 1), "z_raw": xv(xyz_raw, 2),
            "x_sm": xv(xyz_sm, 0), "y_sm": xv(xyz_sm, 1), "z_sm": xv(xyz_sm, 2),
        })
        if len(self.rows) - self._last >= 30:

```

```

        self._save_excel()
        self._last = len(self.rows)

def _save_excel(self):
    from openpyxl import Workbook as WB
    wb = WB(); ws = wb.active; ws.title = "LED5Cam"
    cols = ["sample", "ts",
            "led1_x", "led1_y", "led2_x", "led2_y",
            "led3_x", "led3_y", "led4_x", "led4_y",
            "x_raw", "y_raw", "z_raw", "x_sm", "y_sm", "z_sm"]
    ws.append(cols)
    for r in self.rows:
        ws.append([r[c] for c in cols])
    wb.save(self.excel)

def save(self):
    self._save_excel()
    # Plot
    if self.rows:
        import matplotlib; matplotlib.use("Agg")
        import matplotlib.pyplot as plt
        s = [r["sample"] for r in self.rows if r["x_sm"] is not None]
        xs = [r["x_sm"] for r in self.rows if r["x_sm"] is not None]
        ys = [r["y_sm"] for r in self.rows if r["x_sm"] is not None]
        zs = [r["z_sm"] for r in self.rows if r["x_sm"] is not None]
        if s:
            plt.figure(figsize=(10, 5))
            plt.plot(s, xs, label="X", linewidth=2)
            plt.plot(s, ys, label="Y", linewidth=2)
            plt.plot(s, zs, label="Z", linewidth=2)
            plt.title("LED 3D Position (5-cam)"); plt.xlabel("Sample")
            plt.grid(True, alpha=0.3); plt.legend(); plt.tight_layout()
            plt.savefig(self.plot, dpi=160); plt.close()
        print(f"Saved → {self.excel}")

#
=====
# MAIN
#
=====

def main():
    ap = argparse.ArgumentParser(description="5-cam LED tracker: cam0=MediaPipe, LED1-4=triangulation")
    ap.add_argument("--pose-cam", type=int, default=POSE_CAM_INDEX)
    ap.add_argument("--led-cams", type=int, nargs=4, default=LED_CAM_INDICES,
                    metavar=("A", "B", "C", "D"),
                    help="4 OS camera indices for LED cams (default: 1 2 3 5)")
    ap.add_argument("--primary-pair", type=int, nargs=2, default=[0, 1],
                    metavar=("A", "B"),
                    help="0-based positions in --led-cams list to use for triangulation (default: 0 1)")
    ap.add_argument("--calibration", default="calibration/stereo_calibration.npz")
    ap.add_argument("--output-dir", default="outputs")
    ap.add_argument("--width", type=int, default=1280)
    ap.add_argument("--height", type=int, default=720)
    ap.add_argument("--threshold", type=int, default=220,

```



```

        help="Starting threshold (200-245). Each cam has its own trackbar.")
ap.add_argument("--min-area",    type=float, default=20.0)
ap.add_argument("--smooth",     type=float, default=0.30)
ap.add_argument("--px-smooth",  type=float, default=0.30,
        help="Per-camera pixel EMA smoothing (0=frozen, 1=raw)")
ap.add_argument("--px-hold",    type=int,  default=20,
        help="Frames to hold last known LED position after loss")
args = ap.parse_args()

```

— Calibration

```

calib_path = resolve(args.calibration)
out_dir    = resolve(args.output_dir)
try:
    K1, D1, K2, D2, R, T = load_calibration(calib_path)
    print(f"Calibration loaded: {calib_path}")
except FileNotFoundError as e:
    print(f"[ERROR] {e}")
    return

pa, pb     = args.primary_pair
led_indices = args.led_cams
logger     = Logger(out_dir)

print(f"\nStarting all 5 cameras...")
print(f" Pose cam    : idx={args.pose_cam}")
print(f" LED cams    : {led_indices} (LED1-LED4)")
print(f" Stereo pair  : LED{pa+1}(idx={led_indices[pa]}) + LED{pb+1}(idx={led_indices[pb]})")
print(f" Threshold start: {args.threshold} (tune per-cam with trackbars)\n")

```

— Open cameras

```

pose_ct = CameraThread(cam_id=0, index=args.pose_cam, width=args.width,
height=args.height)
led_cts = [CameraThread(cam_id=i+1, index=idx, width=args.width, height=args.height)
        for i, idx in enumerate(led_indices)]

pose_ct.start()
for ct in led_cts:
    ct.start(); time.sleep(0.6)

all_cts = [pose_ct] + led_cts
deadline = time.time() + 20.0
while time.time() < deadline:
    if all(ct.ok or ct.error for ct in all_cts):
        break
    time.sleep(0.2)

for ct in all_cts:
    if ct.error:
        print(f" !! {ct.error}")

ready = sum(1 for ct in all_cts if ct.ok)
print(f"\n{ready}/5 cameras ready.")
if ready < 3:
    print("ERROR: need at least pose cam + 2 LED cams."); [ct.stop() for ct in all_cts]; return

```

— MediaPipe

```
landmarker = build_landmarker()
frame_ms = 0

# — Windows + per-camera threshold trackbars —————
POSE_WIN = "Cam0 — MediaPipe Pose"
LED_WINS = [f"LED{i+1} idx={led_indices[i]}" for i in range(4)]
BIN_WINS = [f"Binary LED{i+1}" for i in range(4)]
THRESH_KEY = [f"thresh_led{i+1}" for i in range(4)]

cv2.namedWindow(POSE_WIN, cv2.WINDOW_NORMAL)
cv2.resizeWindow(POSE_WIN, DISPLAY_W, DISPLAY_H)
cv2.moveWindow(POSE_WIN, 0, 0)

for i in range(4):
    col, row = i % 2, i // 2
    x = DISPLAY_W + 10 + col * (DISPLAY_W + 10)
    y = row * (DISPLAY_H + 55)
    cv2.namedWindow(LED_WINS[i], cv2.WINDOW_NORMAL)
    cv2.resizeWindow(LED_WINS[i], DISPLAY_W, DISPLAY_H)
    cv2.moveWindow(LED_WINS[i], x, y)
    # BIN window below each LED window
    cv2.namedWindow(BIN_WINS[i], cv2.WINDOW_NORMAL)
    cv2.resizeWindow(BIN_WINS[i], DISPLAY_W, DISPLAY_H // 2)
    cv2.moveWindow(BIN_WINS[i], x, y + DISPLAY_H + 5)
    # Per-camera threshold trackbar ON each LED window
    cv2.createTrackbar("threshold", LED_WINS[i], args.threshold, 255, lambda _: None)

blank = np.zeros((DISPLAY_H, DISPLAY_W, 3), np.uint8)

print("\n[RUNNING] Point phone torch / LED at cameras.")
print(" Each LED window has its OWN threshold trackbar — tune per camera!")
print(" Lower threshold = more sensitive. Start at 200-220 for Logitech cams.")
print(" Press Q to quit.\n")

# Per-camera pixel EMA state
px_smooth = [None] * 4 # [x, y] float
px_lost = [0] * 4 # frames since last detection

xyz_smooth = None
sample_idx = 0

try:
    while True:
        # — Cam0: MediaPipe
```

```
pose_frame = pose_ct.get_frame()
if pose_frame is not None:
    pf = pose_frame.copy()
    h, w = pf.shape[:2]
    if landmarker is not None:
        try:
            frame_ms += 33
            rgb = cv2.cvtColor(pf, cv2.COLOR_BGR2RGB)
            mp_img = mp.Image(image_format=mp.ImageFormat.SRGB, data=rgb)
            res = landmarker.detect_for_video(mp_img, frame_ms)
```

```

        if res.pose_landmarks:
            draw_skeleton(pf, res.pose_landmarks[0], h, w)
        except Exception:
            pass
    cv2.putText(pf, "Cam0 — MediaPipe Pose", (8, 22),
               cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 255), 1)
    pose_disp = cv2.resize(pf, (DISPLAY_W, DISPLAY_H))
else:
    pose_disp = blank.copy()
    cv2.putText(pose_disp, "Cam0 NOT READY", (10, DISPLAY_H//2),
               cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)

```

— LED cams: detect

```

raw_pts = [] # raw detection per cam (None if not found)
bins = []

for i, ct in enumerate(led_cts):
    frame = ct.get_frame()
    thresh = max(cv2.getTrackbarPos("threshold", LED_WINS[i], 50)

    if frame is None:
        raw_pts.append(None); bins.append(None); continue

    center, debug = detect_led(frame, thresh, args.min_area)
    raw_pts.append(center)
    bins.append(debug)

```

— Per-camera pixel EMA smoothing

```

smooth_pts = []
for i, pt in enumerate(raw_pts):
    if pt is not None:
        px_lost[i] = 0
        if px_smooth[i] is None:
            px_smooth[i] = [float(pt[0]), float(pt[1])]
        else:
            a = args.px_smooth
            px_smooth[i][0] = a * pt[0] + (1-a) * px_smooth[i][0]
            px_smooth[i][1] = a * pt[1] + (1-a) * px_smooth[i][1]
            smooth_pts.append((px_smooth[i][0], px_smooth[i][1]))
    else:
        px_lost[i] += 1
        if px_smooth[i] is not None and px_lost[i] <= args.px_hold:
            smooth_pts.append((px_smooth[i][0], px_smooth[i][1]))
        else:
            px_smooth[i] = None
            smooth_pts.append(None)

```

— Triangulate on primary stereo pair

```

xyz_raw = None
if smooth_pts[pa] is not None and smooth_pts[pb] is not None:
    xyz_raw = triangulate(smooth_pts[pa], smooth_pts[pb],
                          K1, D1, K2, D2, R, T)
if xyz_raw is not None:
    if xyz_smooth is None:
        xyz_smooth = xyz_raw.copy()
    else:

```

```

xyz_smooth = args.smooth * xyz_raw + (1-args.smooth) * xyz_smooth

xyz_txt = (
    f"X={xyz_smooth[0]: .3f} Y={xyz_smooth[1]: .3f} Z={xyz_smooth[2]: .3f}"
    if xyz_smooth is not None
    else "XYZ: need LED in stereo pair"
)

if xyz_raw is not None:
    print(f"[{sample_idx:5d}] {xyz_txt}")
    logger.add(sample_idx, raw_pts, xyz_raw, xyz_smooth)
    sample_idx += 1

# — Draw LED windows


---


for i, ct in enumerate(led_cts):
    frame = ct.get_frame()
    color = LED_COLORS[i]
    is_stereo = (i == pa or i == pb)

    if frame is None:
        disp = blank.copy()
        cv2.putText(disp, f"LED{i+1} NOT READY", (10, DISPLAY_H//2),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (0, 0, 255), 2)
    else:
        disp = frame.copy()
        draw_led(disp, smooth_pts[i], color)

    # Detection status
    if smooth_pts[i] is not None:
        det_txt = f"({smooth_pts[i][0]:.0f}, {smooth_pts[i][1]:.0f})"
        det_color = color
    else:
        det_txt = "NO LED"
        det_color = (0, 0, 255)

    pair_tag = " [STEREO]" if is_stereo else " [EXTRA]"
    cv2.putText(disp, f"LED{i+1}{pair_tag} idx={ct.index}({ct.backend})",
                (10, 25), cv2.FONT_HERSHEY_SIMPLEX, 0.50, (255, 255, 0), 1)
    cv2.putText(disp, det_txt,
                (10, 52), cv2.FONT_HERSHEY_SIMPLEX, 0.55, det_color, 2)
    cv2.putText(disp, xyz_txt,
                (10, 78), cv2.FONT_HERSHEY_SIMPLEX, 0.42, (0, 0, 255), 1)

    cv2.imshow(LED_WINS[i], cv2.resize(disp, (DISPLAY_W, DISPLAY_H)))
    if bins[i] is not None:
        b3 = cv2.cvtColor(bins[i], cv2.COLOR_GRAY2BGR) if len(bins[i].shape) == 2 else
bins[i]
        cv2.imshow(BIN_WINS[i], cv2.resize(b3, (DISPLAY_W, DISPLAY_H // 2)))

# XYZ on pose window too
cv2.putText(pose_disp, xyz_txt, (8, DISPLAY_H - 10),
            cv2.FONT_HERSHEY_SIMPLEX, 0.38, (0, 0, 255), 1)
cv2.imshow(POSE_WIN, pose_disp)

if cv2.waitKey(1) & 0xFF == ord("q"):
    print("\nQ — exiting.")

```

```

        break

    finally:
        if landmarker:
            landmarker.close()
        for ct in all_cts:
            ct.stop()
        cv2.destroyAllWindows()
        logger.save()

if __name__ == "__main__":
    main()

```

ANNEXURE B– Single Camera Clibartion.py

```

import glob, os, threading, time
import cv2, numpy as np

```

```

#

```

```

# CONFIG ← all hyperparameters live here, nowhere else
#

```

```

CHECKERBOARD    = (7, 5)
SQUARE_SIZE     = 1.0
CAP_INTERVAL    = 1.0
CAP_TARGET      = 60      # frames per camera
OUT_DIR         = "stereo_images"

```

```

# — Display

```

```

—
DISPLAY_W, DISPLAY_H = 400, 300

```

```

# — Detection smoothing

```

```

SMOOTH_N        = 5
STABLE_THRESH   = 0.4

```

```

# — Corner-subpix

```

```

SUBPIX_WIN      = (11,11)
SUBPIX_ZERO_ZONE = (-1, -1)
SUBPIX_CRIT     = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 50,
0.0001)

```

```

# — CLAHE

```

```

—
CLAHE_CLIP      = 3.0
CLAHE_GRID      = (8, 8)

```

```

# — Calibration

```

```

CALIB_FLAGS     = cv2.CALIB_RATIONAL_MODEL

```

```
OUTLIER_PERCENTILE = 75
MIN_CALIB_FRAMES = 5
```

```
# — Camera index (None = prompt user to pick) —————
CAM_INDEX = None
```

```
#
```

```
=====
# AUTO-DETECT: scan indices, show user what's available
```

```
#
```

```
=====
def detect_cameras():
    print("\n Scanning for cameras...")
    found = []
    for idx in range(9):
        for api, name in [(cv2.CAP_DSHOW, "dshow"), (cv2.CAP_MSMF, "msmf")]:
            cap = cv2.VideoCapture(idx, api)
            if not cap.isOpened():
                cap.release(); continue
            cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
            cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
            ok = False
            for _ in range(30):
                ret, fr = cap.read()
                if ret and fr is not None and fr.size > 0 and float(np.mean(fr)) > 1.0:
                    ok = True; break
            cap.release()
            if ok:
                found.append(idx)
                print(f"  idx={idx} OK ({name})")
                break
            else:
                continue

    found = sorted(set(found))
    print(f" All working indices: {found}")

    if len(found) == 0:
        print(" ERROR: no cameras found!")
        return None

    return found
```

```
#
```

```
=====
# CAMERA THREAD
```

```
#
```

```
=====
class CameraThread:
    def __init__(self, index):
        self.index = index
        self.frame = None; self.ok = False; self.error = None; self.backend = "?"
        self._lock = threading.Lock(); self._stop = threading.Event()
```

```

self._thread = threading.Thread(target=self._run, daemon=True)

def start(self): self._thread.start()
def stop(self): self._stop.set(); self._thread.join(timeout=3)
def get_frame(self):
    with self._lock: return self.frame.copy() if self.frame is not None else None

def _open(self):
    for api, bname in
[(cv2.CAP_DSHOW,"dshow"),(cv2.CAP_MSMF,"msmf"),(cv2.CAP_ANY,"any")]:
        for _ in range(3):
            cap = cv2.VideoCapture(self.index, api)
            if not cap.isOpened(): cap.release(); time.sleep(0.3); continue
            cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
            cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
            cap.set(cv2.CAP_PROP_FPS, 30)
            for _ in range(60):
                ret, fr = cap.read()
                if ret and fr is not None and fr.size > 0 and float(np.mean(fr)) > 1.0:
                    self.backend = bname; return cap
            cap.release(); time.sleep(0.4)
    return None

def _run(self):
    cap = None
    for attempt in range(4):
        cap = self._open()
        if cap: break
        print(f" cam (idx={self.index}) attempt {attempt+1} failed, retrying...")
        time.sleep(1.0)
    if cap is None:
        self.error = f"cam idx={self.index}: failed"; return
    self.ok = True
    print(f" cam READY idx={self.index} ({self.backend})")
    while not self._stop.is_set():
        ret, frame = cap.read()
        if ret and frame is not None and frame.size > 0:
            with self._lock: self.frame = frame
        else: time.sleep(0.005)
    cap.release()

```

#

CHECKER THREAD

#

class CheckerThread:

```

def __init__(self, cam_thread, board):
    self.cam = cam_thread; self.board = board
    self.result = (0.0, None, None, None)
    self._lock = threading.Lock(); self._stop = threading.Event()
    self._thread = threading.Thread(target=self._run, daemon=True)
    self._history = []

```

```

def start(self): self._thread.start()

```

```

def stop(self): self._stop.set(); self._thread.join(timeout=2)
def get(self):
    with self._lock: return self.result

def _detect_on_gray(self, gray):
    clahe = cv2.createCLAHE(clipLimit=CLAHE_CLIP, tileGridSize=CLAHE_GRID)
    gray_eq = clahe.apply(gray)
    for img in [gray_eq, gray]:
        ret, corners = cv2.findChessboardCornersSB(
            img, self.board, cv2.CALIB_CB_EXHAUSTIVE | cv2.CALIB_CB_ACCURACY)
        if ret:
            corners = cv2.cornerSubPix(gray, corners, SUBPIX_WIN, SUBPIX_ZERO_ZONE,
SUBPIX_CRIT)
            return True, corners
        ret, corners = cv2.findChessboardCorners(
            img, self.board,
            cv2.CALIB_CB_ADAPTIVE_THRESH | cv2.CALIB_CB_NORMALIZE_IMAGE)
        if ret:
            corners = cv2.cornerSubPix(gray, corners, SUBPIX_WIN, SUBPIX_ZERO_ZONE,
SUBPIX_CRIT)
            return True, corners
    return False, None

def _run(self):
    while not self._stop.is_set():
        frame = self.cam.get_frame()
        if frame is None: time.sleep(0.02); continue
        h, w = frame.shape[:2]
        detected = False; best_corners = None; ann = None; centroid = None

        for sf in [1.0, 0.5, 0.35]:
            nw, nh = int(w*sf), int(h*sf)
            if nw < 80 or nh < 60: continue
            resized = cv2.resize(frame, (nw, nh)) if sf < 1.0 else frame.copy()
            gray = cv2.cvtColor(resized, cv2.COLOR_BGR2GRAY)
            ret, corners = self._detect_on_gray(gray)
            if ret:
                if sf < 1.0: corners = corners / sf
                detected = True; best_corners = corners
                centroid = corners.mean(axis=0)[0]
                ann_frame = cv2.resize(frame, (DISPLAY_W, DISPLAY_H))
                sx, sy = DISPLAY_W/w, DISPLAY_H/h
                dc = corners.copy()
                dc[:,0] *= sx; dc[:,1] *= sy
                cv2.drawChessboardCorners(ann_frame, self.board, dc, True)
                ann = ann_frame
                break

        self._history.append(detected)
        if len(self._history) > SMOOTH_N: self._history.pop(0)
        score = sum(self._history) / len(self._history)
        if ann is None: ann = cv2.resize(frame, (DISPLAY_W, DISPLAY_H))

        with self._lock:
            self.result = (score, best_corners if detected else None, ann, centroid)
            time.sleep(0.04)

```


#

COVERAGE GRID

#

```
class CoverageGrid:
    def __init__(self, w, h, cols=3, rows=3):
        self.w = w; self.h = h; self.cols = cols; self.rows = rows
        self.covered = [[False]*cols for _ in range(rows)]

    def update(self, cx, cy):
        col = min(int(cx / self.w * self.cols), self.cols-1)
        row = min(int(cy / self.h * self.rows), self.rows-1)
        self.covered[row][col] = True

    def coverage_pct(self):
        total = self.cols * self.rows
        done = sum(1 for r in self.covered for c in r if c)
        return int(100 * done / total)

    def draw(self, canvas, ox, oy, bw, bh):
        cw = bw // self.cols; ch = bh // self.rows
        for r in range(self.rows):
            for c in range(self.cols):
                x1, y1 = ox + c*cw, oy + r*ch
                x2, y2 = x1+cw-2, y1+ch-2
                col = (0,180,60) if self.covered[r][c] else (60,60,60)
                cv2.rectangle(canvas, (x1,y1), (x2,y2), col, -1)
                cv2.rectangle(canvas, (x1,y1), (x2,y2), (150,150,150), 1)
```

#

PHASE 1 — CAPTURE (single cam)

#

```
def phase_capture(cam_idx, target, interval):
    # Each camera gets its own folder: stereo_images/cam{idx}/
    folder = os.path.join(OUT_DIR, f"cam{cam_idx}")
    os.makedirs(folder, exist_ok=True)

    print(f"\nOpening camera idx={cam_idx} ...")
    ct = CameraThread(index=cam_idx)
    ct.start(); time.sleep(0.6)

    deadline = time.time() + 20.0
    while time.time() < deadline:
        if ct.ok or ct.error: break
        time.sleep(0.2)

    if ct.error:
        print(f" !! {ct.error}")
    if not ct.ok:
        print("ERROR: camera failed to open.")
```

```

ct.stop(); return False

cht = CheckerThread(ct, CHECKERBOARD)
cht.start(); time.sleep(0.8)

WIN = f"Cam{cam_idx}"
cv2.namedWindow(WIN, cv2.WINDOW_NORMAL)
cv2.resizeWindow(WIN, DISPLAY_W, DISPLAY_H)
cv2.moveWindow(WIN, 0, 0)

blank      = np.zeros((DISPLAY_H, DISPLAY_W, 3), np.uint8)
capture_count = 0
last_cap_time = 0.0
grid       = CoverageGrid(640, 480)

print(f'{'='*55}')
print(f" CAPTURING cam{cam_idx} — board:
{CHECKERBOARD[0]}x{CHECKERBOARD[1]} target: {target}")
print(f" MOVE BOARD: corners, edges, center, tilted!")
print(f" Auto-saves every {interval}s | Q to stop early")
print(f" Saving to: {folder}")
print(f'{'='*55}\n")

try:
    while True:
        now = time.time()
        raw_frame = ct.get_frame()
        score, corners, ann_frame, centroid = cht.get()

        disp = ann_frame if ann_frame is not None else blank.copy()
        disp = cv2.resize(disp, (DISPLAY_W, DISPLAY_H))

        if score >= STABLE_THRESH:
            col = (0, 255, 0); status = f"DETECTED ({int(score*100)}%)"
        elif score > 0:
            col = (0, 200, 255); status = f"WEAK ({int(score*100)}%)"
        else:
            col = (0, 0, 255); status = "NO BOARD"

        elapsed = now - last_cap_time
        countdown = max(0.0, interval - elapsed)

        cv2.putText(disp, f"Cam{cam_idx} | {capture_count}/{target}",
            (8,24), cv2.FONT_HERSHEY_SIMPLEX, 0.55, col, 2)
        cv2.putText(disp, status, (8,48),
            cv2.FONT_HERSHEY_SIMPLEX, 0.50, col, 2)
        cv2.putText(disp, f"Save: {countdown:.1f}s", (8,68),
            cv2.FONT_HERSHEY_SIMPLEX, 0.42, (0,200,255), 1)
        cov = grid.coverage_pct()
        cv2.putText(disp, f"Coverage: {cov}%", (8,88),
            cv2.FONT_HERSHEY_SIMPLEX, 0.42,
            (0,255,0) if cov >= 70 else (0,180,255), 1)
        grid.draw(disp, DISPLAY_W-70, 4, 66, 66)

        bar_w = int(DISPLAY_W * capture_count / target)
        cv2.rectangle(disp, (0, DISPLAY_H-18), (DISPLAY_W, DISPLAY_H), (30,30,30), -1)
        cv2.rectangle(disp, (0, DISPLAY_H-18), (bar_w, DISPLAY_H), (0,180,60), -1)

```

```

cv2.putText(dis, f"SAVED: {capture_count}/{target}",
            (6, DISPLAY_H-4), cv2.FONT_HERSHEY_SIMPLEX, 0.38, (255,255,255), 1)
cv2.imshow(WIN, disp)

# Auto-save every interval seconds regardless of detection
if raw_frame is not None and elapsed >= interval:
    last_cap_time = now
    cv2.imwrite(os.path.join(folder, f"frame_{capture_count:04d}.png"), raw_frame)
    capture_count += 1
    if centroid is not None:
        grid.update(centroid[0], centroid[1])
        tag = "DETECTED" if score >= STABLE_THRESH else "no-board"
        print(f" cam{cam_idx} [{capture_count:>3d}/{target}] {tag} cov={grid.coverage_pct()}%")

    if capture_count >= target:
        print(f"\n cam{cam_idx} hit target {target}!"); break

    if cv2.waitKey(1) & 0xFF == ord("q"):
        print(" Q — stopping."); break

finally:
    cv2.destroyAllWindows()
    cht.stop()
    ct.stop()

print(f"\n Done. cam{cam_idx}={capture_count} frames saved to {folder}\n")
return True

```

#

PHASE 2 — CALIBRATION (single cam)

#

```

def fmt_matrix(mat):
    lines = []
    if mat.ndim <= 1:
        lines.append(" " + " ".join(f"{v:.8f}" for v in mat.flatten()))
    else:
        for row in mat:
            lines.append(" " + " ".join(f"{v:14.8f}" for v in row))
    return "\n".join(lines)

def per_frame_err(objp, corners, rvec, tvec, mtx, dist):
    proj, _ = cv2.projectPoints(objp, rvec, tvec, mtx, dist)
    return cv2.norm(corners, proj, cv2.NORM_L2) / np.sqrt(len(objp))

def calibrate_single(cam_idx):
    folder = os.path.join(OUT_DIR, f"cam{cam_idx}")
    # Results go into stereo_images/cam{idx}/calibration/
    calib_dir = os.path.join(folder, "calibration")
    os.makedirs(calib_dir, exist_ok=True)

    print(f"\n{' '*55}")
    print(f" CALIBRATION — cam{cam_idx}")
    print(f" Images : {folder}")

```

```

print(f" Output : {calib_dir}")
print(f"{'='*55}\n")

objp = np.zeros((CHECKERBOARD[0]*CHECKERBOARD[1], 3), np.float32)
objp[:, :2] = np.mgrid[0:CHECKERBOARD[0], 0:CHECKERBOARD[1]].T.reshape((-1,2))
objp *= SQUARE_SIZE
clahe = cv2.createCLAHE(clipLimit=CLAHE_CLIP, tileGridSize=CLAHE_GRID)

imgs = sorted(glob.glob(os.path.join(folder, "*.png")))
if not imgs:
    print(f" ERROR: no images found in {folder}\n"); return

print(f" Step 1 — detecting corners in {len(imgs)} images...")
found_list, corner_list = [], []
for f in imgs:
    img = cv2.imread(f)
    if img is None: found_list.append(False); continue
    gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
    gray_eq = clahe.apply(gray)
    ret, corners = cv2.findChessboardCornersSB(
        gray_eq, CHECKERBOARD, cv2.CALIB_CB_EXHAUSTIVE | cv2.CALIB_CB_ACCURACY)
    if not ret:
        ret, corners = cv2.findChessboardCornersSB(
            gray, CHECKERBOARD, cv2.CALIB_CB_EXHAUSTIVE | cv2.CALIB_CB_ACCURACY)
    if not ret:
        ret, corners = cv2.findChessboardCorners(
            gray_eq, CHECKERBOARD,
            cv2.CALIB_CB_ADAPTIVE_THRESH | cv2.CALIB_CB_NORMALIZE_IMAGE)
    if ret:
        corners = cv2.cornerSubPix(gray, corners, SUBPIX_WIN, SUBPIX_ZERO_ZONE,
SUBPIX_CRIT)
        corner_list.append((corners, gray.shape[::-1]))
        found_list.append(True)
    else:
        found_list.append(False)

pct = int(100*sum(found_list)/max(len(imgs),1))
print(f" cam{cam_idx}: {sum(found_list)}/{len(imgs)} images detected ({pct}%)")

if len(corner_list) < MIN_CALIB_FRAMES:
    print(f" ERROR: only {len(corner_list)} frames — need at least {MIN_CALIB_FRAMES}.
Recapture.\n"); return

print(f"\n Step 2 — calibrating + outlier filter...")
objpts = [objp]*len(corner_list)
imgpts = [d[0] for d in corner_list]
shape = corner_list[0][1]

rms1, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(
    objpts, imgpts, shape, None, None, flags=CALIB_FLAGS)
print(f" pass1 RMS={rms1:.4f} ({len(corner_list)} frames)")

errs = [per_frame_err(objp, imgpts[j], rvecs[j], tvecs[j], mtx, dist) for j in range(len(corner_list))]
thresh = np.percentile(errs, OUTLIER_PERCENTILE)
keep = [j for j, e in enumerate(errs) if e <= thresh]

objpts2 = [objpts[j] for j in keep]

```

```

imgpts2 = [imgpts[j] for j in keep]
rms2, mtx2, dist2, _, _ = cv2.calibrateCamera(
    objpts2, imgpts2, shape, None, None, flags=CALIB_FLAGS)
print(f" pass2 RMS={rms2:.4f} ({len(keep)} frames after outlier filter)")

tag = "GOOD ✓" if rms2 < 1.0 else ("OK ✓" if rms2 < 3.0 else "HIGH RMS ⚠")

npz_path = os.path.join(calib_dir, f"cam{cam_idx}_calibration.npz")
np.savez(npz_path, cameraMatrix=mtx2, distCoeffs=dist2, rms=np.float64(rms2))

sep = "-"*55
print(f"\n{sep}")
print(f" cam{cam_idx} RMS = {rms2:.4f} [{tag}]")
print(sep)
print(f" cameraMatrix:\n{fmt_matrix(mtx2)}")
print(f" distCoeffs:\n{fmt_matrix(dist2)}")

lines = [
    f"CALIBRATION RESULTS cam{cam_idx} — tubakhxn", "="*55,
    f"Checkerboard: {CHECKERBOARD[0]}x{CHECKERBOARD[1]} | Frames used:
{len(keep)}/{len(corner_list)}",
    "", sep, f"cam{cam_idx} RMS = {rms2:.4f} [{tag}]", sep,
    "cameraMatrix:", fmt_matrix(mtx2),
    "distCoeffs:", fmt_matrix(dist2),
    "", f"Saved → {npz_path}",
]
txt_path = os.path.join(calib_dir, f"cam{cam_idx}_calibration_results.txt")
with open(txt_path, "w", encoding="utf-8") as f: f.write("\n".join(lines))

print(f"\n .npz → {npz_path}")
print(f" .txt → {txt_path}")
print(f"{'='*55}\n")

```

#

MAIN

#

```

def main():
    print("\n" + "="*55)
    print(" SINGLE-CAM CALIBRATION — tubakhxn v5")
    print("="*55)

    # Pick camera index
    cam_idx = CAM_INDEX
    if cam_idx is None:
        available = detect_cameras()
        if available is None: return
        if len(available) == 1:
            cam_idx = available[0]
            print(f" Only one camera found — using idx={cam_idx}")
        else:
            print(f"\n Available cameras: {available}")
            while True:
                try:

```

```

        cam_idx = int(input(f" Enter camera index to calibrate: ").strip())
        if cam_idx in available: break
        print(f" Please enter one of {available}")
    except ValueError:
        print(" Enter a number.")
else:
    print(f" Using manually set CAM_INDEX: {cam_idx}")

folder = os.path.join(OUT_DIR, f"cam{cam_idx}")
os.makedirs(folder, exist_ok=True)

print(f"\n Camera      : idx={cam_idx}")
print(f" Checkerboard : {CHECKERBOARD[0]}x{CHECKERBOARD[1]} inner corners")
print(f" Image folder : {folder}")
print(f" Interval: {CAP_INTERVAL}s | Target: {CAP_TARGET} frames")
print("="*55)

count = len(glob.glob(os.path.join(folder, "*.png")))
existing = count > 0

if existing:
    print(f"\n Found {count} existing images in {folder}")
    print("\n [1] Use existing images and calibrate now")
    print(" [2] Capture NEW images (opens camera)")
    while True:
        choice = input("\n Enter 1 or 2: ").strip()
        if choice in ("1", "2"): break
        print(" Please enter 1 or 2.")
else:
    print("\n No existing images — will capture new ones.")
    choice = "2"

if choice == "2":
    ok = phase_capture(cam_idx, CAP_TARGET, CAP_INTERVAL)
    if not ok: return

calibrate_single(cam_idx)

if __name__ == "__main__":
    main()

```

ANNEXURE C– 4 Camera Clibartion.py

```

import argparse
import glob
import itertools
import os
import sys

import cv2
import numpy as np
import matplotlib
matplotlib.use("Agg")      # headless — saves plots as PNG instead of popping a window
import matplotlib.pyplot as plt

```

```
#
```

```
# DEFAULTS (edit here to change globally)
```

```
#
```

```
DEFAULT_BOARD = (7, 5)      # inner corners (cols × rows)
DEFAULT_SQ_SIZE = 1.0       # square size in mm (1.0 = dimensionless)
DEFAULT_OUT_DIR = "calibration_images"
NUM_CAMS      = 4
```

```
#
```

```
# STEP 1 — FIND CORNERS IN EVERY IMAGE, PER CAMERA
```

```
#
```

```
def find_corners_all_cams(cam_folders, board, sq_size):
```

```
    """
```

```
    Returns:
```

```
    objp      — (N,3) float32 object-point template
    per_cam    — list of 4 lists, each containing (corners, img_shape)
                  only for frames where the board WAS found
    frame_masks — list of 4 bool lists (True = board found)
```

```
    """
```

```
    objp = np.zeros((board[0] * board[1], 3), np.float32)
    objp[:, :2] = np.mgrid[0:board[0], 0:board[1]].T.reshape(-1, 2)
    objp *= sq_size
```

```
    per_cam = [[] for _ in range(NUM_CAMS)]
    frame_masks = [] # one list-of-bool per cam
```

```
    # Collect sorted image lists
```

```
    img_lists = []
    for folder in cam_folders:
        imgs = sorted(glob.glob(os.path.join(folder, "*.png")) +
                      glob.glob(os.path.join(folder, "*.jpg")))
        img_lists.append(imgs)
        print(f" {folder}: {len(imgs)} images")
```

```
    n_frames = min(len(il) for il in img_lists)
```

```
    if n_frames == 0:
```

```
        sys.exit("ERROR: No images found in one or more camera folders.")
```

```
    print(f" → Using {n_frames} synchronized frames per camera\n")
```

```
    for cam_i in range(NUM_CAMS):
```

```
        found = []
```

```
        print(f" Detecting corners — cam{cam_i} ...", end="", flush=True)
```

```
        for j in range(n_frames):
```

```
            img = cv2.imread(img_lists[cam_i][j])
```

```
            if img is None:
```

```
                found.append(False)
```

```
                continue
```

```
            gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
```

```

ret, corners = cv2.findChessboardCornersSB(
    gray, board, flags=cv2.CALIB_CB_EXHAUSTIVE | cv2.CALIB_CB_ACCURACY)
if ret:
    per_cam[cam_i].append((corners, gray.shape[:-1]))
    found.append(True)
    print(".", end="", flush=True)
else:
    found.append(False)
    print("x", end="", flush=True)
frame_masks.append(found)
print(f" {sum(found)}/{n_frames} found")

```

```

return objp, per_cam, frame_masks, img_lists, n_frames

```

```

#

```

```

# STEP 2 — CALIBRATE EACH CAMERA INDIVIDUALLY (intrinsics)
#

```

```

def calibrate_intrinsics(cam_i, objp, per_cam, out_dir):
    data = per_cam[cam_i]
    if len(data) < 5:
        print(f" cam{cam_i}: only {len(data)} valid frames — skipping intrinsics")
        return None, None, None

    img_shape = data[0][1]
    objpoints = [objp] * len(data)
    imgpoints = [d[0] for d in data]

    rms, mtx, dist, rvecs, tvecs = cv2.calibrateCamera(
        objpoints, imgpoints, img_shape, None, None)

    out_path = os.path.join(out_dir, f"intrinsics_cam{cam_i}.npz")
    np.savez(out_path, cameraMatrix=mtx, distCoeffs=dist, rms=np.float64(rms))
    print(f" cam{cam_i} intrinsics RMS={rms:.4f} → {os.path.basename(out_path)}")
    return mtx, dist, rms

```

```

#

```

```

# STEP 3 — STEREO CALIBRATE ONE PAIR
#

```

```

def calibrate_stereo_pair(cam_a, cam_b, objp, per_cam,
    mtx_a, dist_a, mtx_b, dist_b,
    frame_masks, n_frames, out_dir):
    """
    Builds synchronized objpoints / imgpoints only for frames where BOTH
    cameras found the board.
    """
    # Find frames where both cams detected the board
    shared_frames = [

```



```

    i for i in range(n_frames)
    if frame_masks[cam_a][i] and frame_masks[cam_b][i]
]

# Map frame index → position in per_cam[x] (only detected frames stored)
def frame_to_idx(cam_i, frame_i):
    count = 0
    for k in range(frame_i):
        if frame_masks[cam_i][k]:
            count += 1
    return count

print(f"\n Pair cam{cam_a}–cam{cam_b}: {len(shared_frames)} shared frames", end="")

if len(shared_frames) < 5:
    print(f" [SKIP] need ≥5, got {len(shared_frames)}")
    return None

objpoints = []
pts_a, pts_b = [], []
for fi in shared_frames:
    idx_a = frame_to_idx(cam_a, fi)
    idx_b = frame_to_idx(cam_b, fi)
    objpoints.append(objp)
    pts_a.append(per_cam[cam_a][idx_a][0])
    pts_b.append(per_cam[cam_b][idx_b][0])

img_shape = per_cam[cam_a][0][1]

stereo_crit = (cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER, 200, 1e-7)
rms, cM1, dC1, cM2, dC2, R, T, E, F = cv2.stereoCalibrate(
    objpoints, pts_a, pts_b,
    mtx_a, dist_a, mtx_b, dist_b,
    img_shape,
    criteria=stereo_crit,
    flags=cv2.CALIB_FIX_INTRINSIC
)

out_path = os.path.join(out_dir, f"stereo_cam{cam_a}_cam{cam_b}.npz")
np.savez(out_path,
    cameraMatrix1=cM1, distCoeffs1=dC1,
    cameraMatrix2=cM2, distCoeffs2=dC2,
    R=R, T=T, E=E, F=F,
    rms=np.float64(rms))

tag = "✓ GOOD" if rms < 1.0 else "⚠ HIGH"
print(f" {tag} RMS={rms:.4f} → {os.path.basename(out_path)}")
return rms

```

#

STEP 4 — PLOT SUMMARY

#

```

def plot_summary(pair_results, intrinsic_rms, out_dir):
    labels_stereo = [f"cam{a}-cam{b}" for (a, b), r in pair_results.items() if r is not None]
    rms_stereo = [r for (a, b), r in pair_results.items() if r is not None]

    fig, axes = plt.subplots(1, 2, figsize=(13, 5))

    # Stereo RMS bar chart
    colors = ["#2ecc71" if r < 1.0 else "#e74c3c" for r in rms_stereo]
    axes[0].bar(labels_stereo, rms_stereo, color=colors, edgecolor="black")
    axes[0].axhline(1.0, color="red", linestyle="--", linewidth=1.2, label="RMS=1.0 threshold")
    axes[0].set_title("Stereo Calibration RMS per Pair")
    axes[0].set_ylabel("RMS reprojection error (px)")
    axes[0].set_xlabel("Camera pair")
    axes[0].legend()
    axes[0].tick_params(axis="x", rotation=30)

    # Individual intrinsic RMS
    cam_labels = [f"cam{i}" for i in range(NUM_CAMS) if intrinsic_rms[i] is not None]
    rms_vals = [r for r in intrinsic_rms if r is not None]
    axes[1].bar(cam_labels, rms_vals, color="#3498db", edgecolor="black")
    axes[1].axhline(1.0, color="red", linestyle="--", linewidth=1.2, label="RMS=1.0 threshold")
    axes[1].set_title("Individual Camera Intrinsic RMS")
    axes[1].set_ylabel("RMS reprojection error (px)")
    axes[1].set_xlabel("Camera")
    axes[1].legend()

    plt.tight_layout()
    plot_path = os.path.join(out_dir, "calibration_summary.png")
    plt.savefig(plot_path, dpi=120)
    plt.close()
    print(f"\n Summary plot → {plot_path}")

#
=====
=====
# MAIN
#
=====
=====

def main():
    ap = argparse.ArgumentParser(
        description="4-camera stereo calibration — all 6 pairs")
    ap.add_argument("--cam0", type=str, default=None)
    ap.add_argument("--cam1", type=str, default=None)
    ap.add_argument("--cam2", type=str, default=None)
    ap.add_argument("--cam3", type=str, default=None)
    ap.add_argument("--out-dir", type=str, default=os.path.join(DEFAULT_OUT_DIR, "calibration"))
    ap.add_argument("--board", type=int, nargs=2, default=list(DEFAULT_BOARD),
        metavar=("COLS", "ROWS"),
        help="Inner corner count e.g. --board 7 5")
    ap.add_argument("--sq-size", type=float, default=DEFAULT_SQ_SIZE,
        help="Physical square size in mm (default 1.0 = dimensionless)")
    args = ap.parse_args()

    board = tuple(args.board)
    sq_size = args.sq_size

```

```

out_dir = args.out_dir
os.makedirs(out_dir, exist_ok=True)

# Resolve camera folders
cam_folders = []
for i, explicit in enumerate([args.cam0, args.cam1, args.cam2, args.cam3]):
    if explicit:
        cam_folders.append(explicit)
    else:
        cam_folders.append(os.path.join(DEFAULT_OUT_DIR, f"cam{i}"))

print("=" * 60)
print(" QUAD CAMERA STEREO CALIBRATION")
print("=" * 60)
print(f" Checkerboard : {board[0]}x{board[1]} inner corners")
print(f" Square size : {sq_size} mm")
print(f" Output dir : {out_dir}")
print(f"\n Camera folders:")
for i, f in enumerate(cam_folders):
    print(f"   cam{i} → {f}")
print()

# — Step 1: Detect corners


---


print("[ STEP 1 ] Detecting checkerboard corners ...")
objp, per_cam, frame_masks, img_lists, n_frames = find_corners_all_cams(
    cam_folders, board, sq_size)

# — Step 2: Individual intrinsics


---


print("\n[ STEP 2 ] Calibrating individual cameras (intrinsics) ...")
mtx = [None] * NUM_CAMS
dist = [None] * NUM_CAMS
i_rms = [None] * NUM_CAMS
for i in range(NUM_CAMS):
    mtx[i], dist[i], i_rms[i] = calibrate_intrinsics(i, objp, per_cam, out_dir)

# Abort if any camera has no intrinsics
missing = [i for i in range(NUM_CAMS) if mtx[i] is None]
if missing:
    print(f"\n WARNING: cam(s) {missing} have no intrinsics. "
          f"Pairs involving them will be skipped.")

# — Step 3: All 6 stereo pairs


---


print("\n[ STEP 3 ] Stereo calibrating all 6 pairs ...")
pairs = list(itertools.combinations(range(NUM_CAMS), 2))
pair_results = {}

for a, b in pairs:
    if mtx[a] is None or mtx[b] is None:
        print(f"\n Pair cam{a}-cam{b}: SKIPPED (missing intrinsics)")
        pair_results[(a, b)] = None
        continue
    rms = calibrate_stereo_pair(
        a, b, objp, per_cam,
        mtx[a], dist[a], mtx[b], dist[b],

```

```

    frame_masks, n_frames, out_dir)
pair_results[(a, b)] = rms

```

— Step 4: Summary

```

print("\n" + "=" * 60)
print(" CALIBRATION SUMMARY")
print("=" * 60)

print("\n Individual intrinsics:")
for i in range(NUM_CAMS):
    if i_rms[i] is not None:
        tag = "✓" if i_rms[i] < 1.0 else "⚠"
        print(f"   cam{i} {tag} RMS={i_rms[i]:.4f}")
    else:
        print(f"   cam{i} ✗ not calibrated")

print("\n Stereo pairs:")
good = 0
for (a, b), rms in pair_results.items():
    if rms is None:
        print(f"   cam{a}-cam{b} ✗ skipped")
    else:
        tag = "✓ GOOD" if rms < 1.0 else "⚠ HIGH RMS"
        print(f"   cam{a}-cam{b} {tag} RMS={rms:.4f}")
        good += 1

print(f"\n {good}/{len(pairs)} stereo pairs calibrated.")
print(f" Files saved to: {out_dir}")

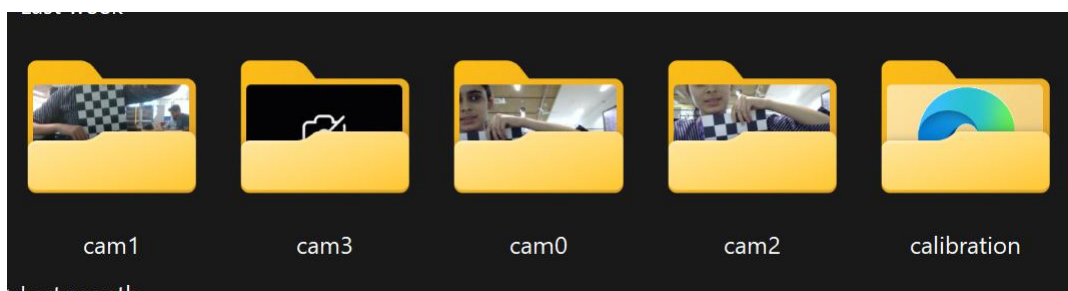
# Plot
try:
    plot_summary(pair_results, i_rms, out_dir)
except Exception as e:
    print(f" [plot skipped: {e}]")

print("=" * 60 + "\n")

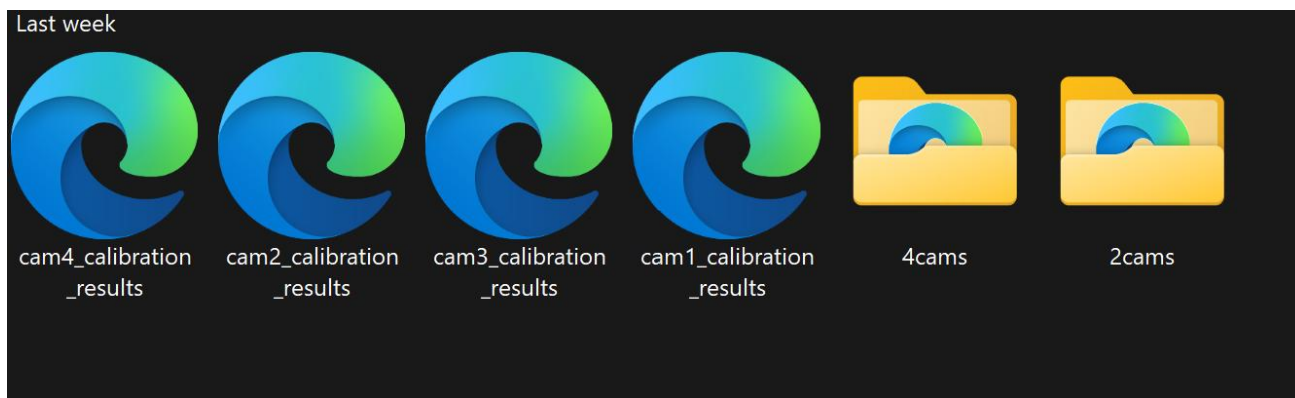
if __name__ == "__main__":
    main()

```

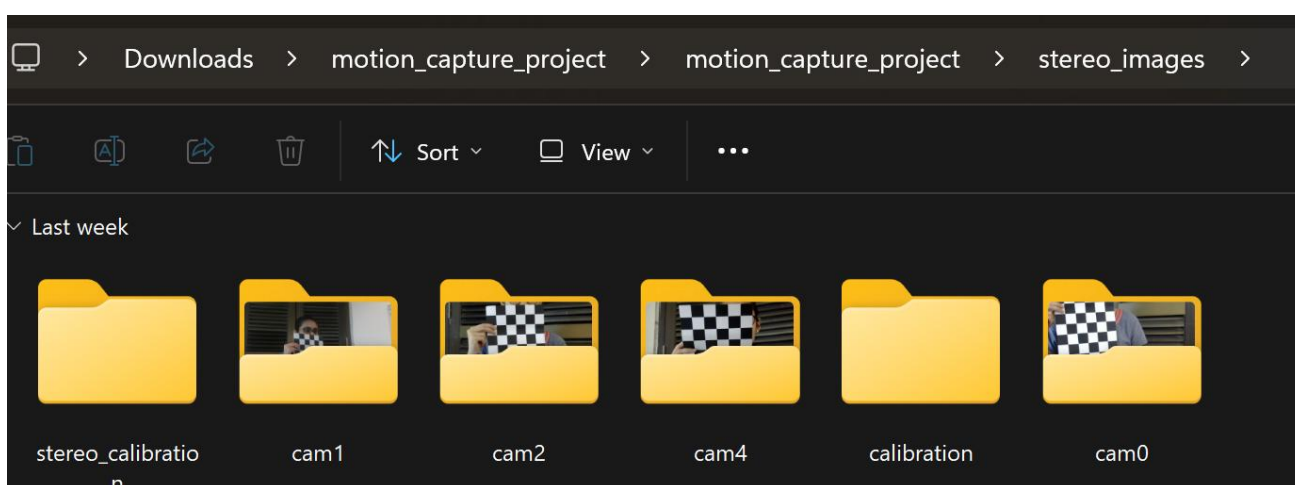
Results Folder:



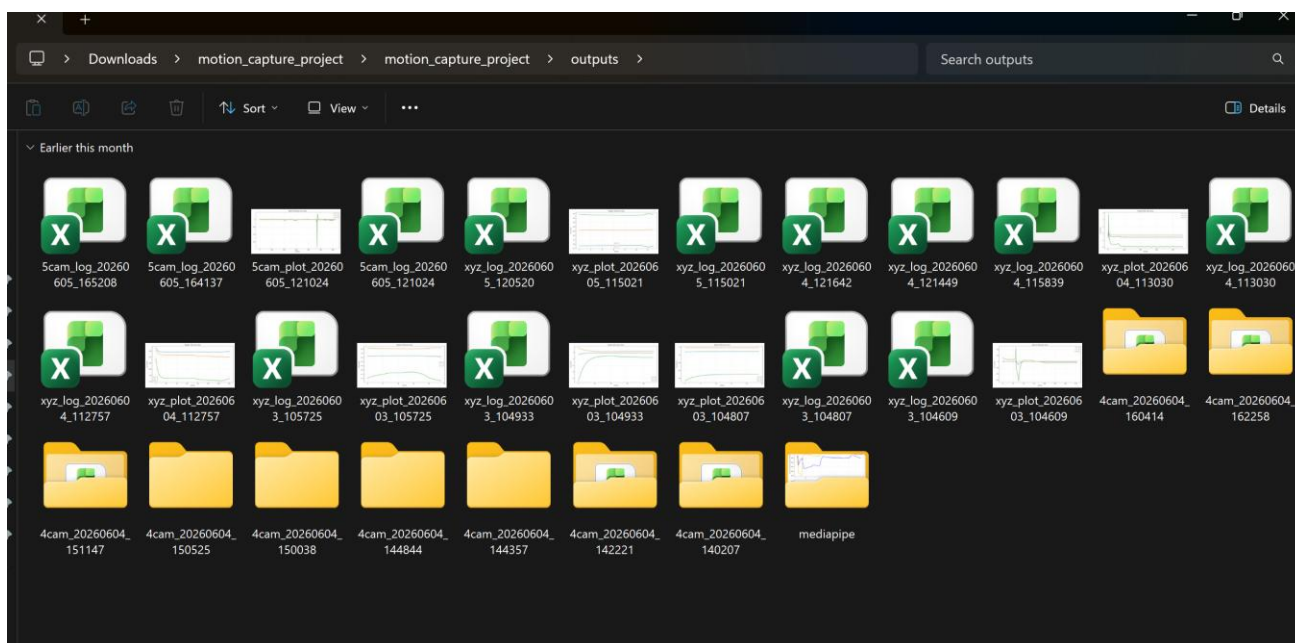
For Each Camera Folder Based



For Stereo Images:

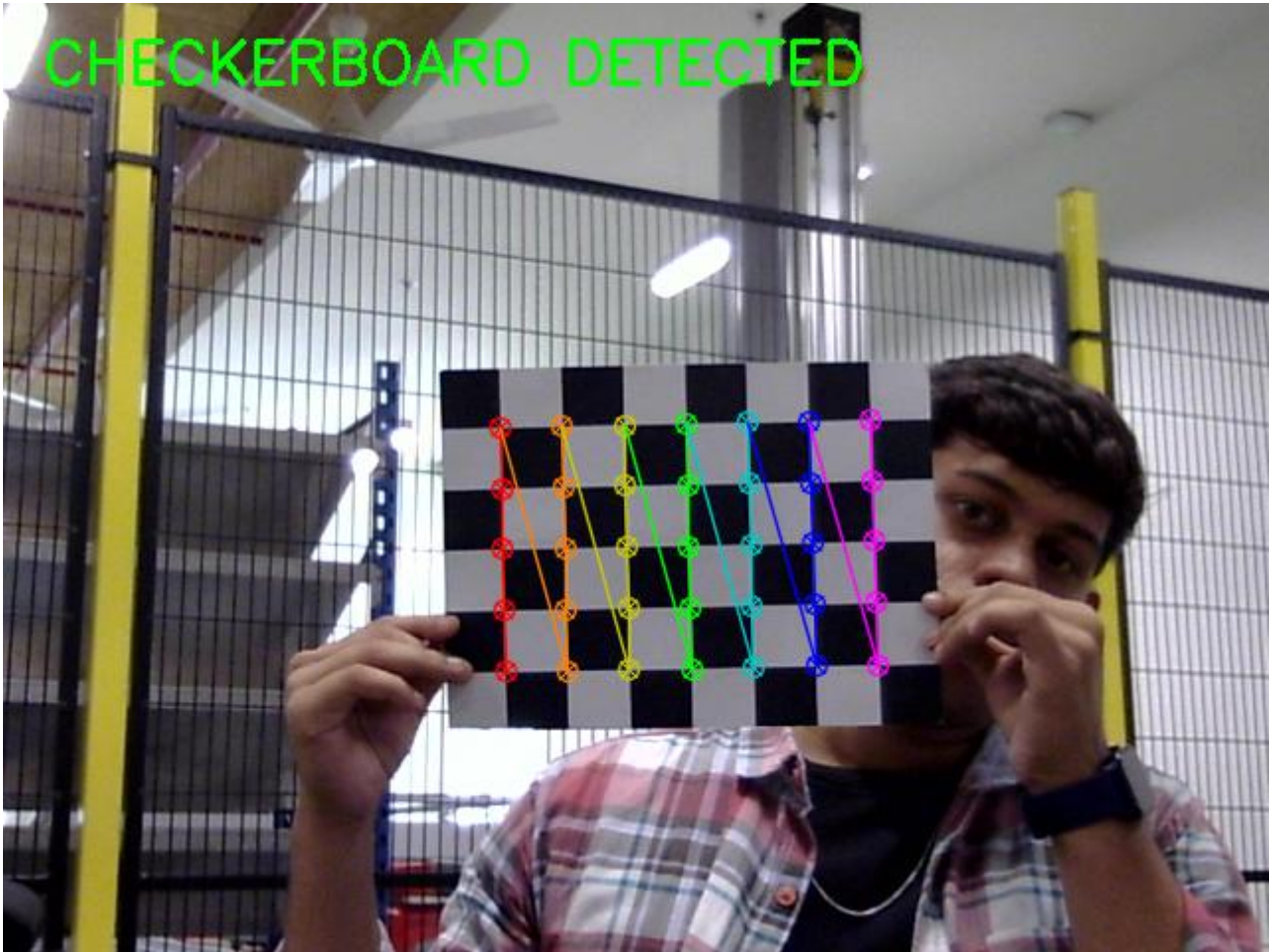


Output Folder With the Graphs and Excel Sheets:



3.4 Stereo Calibration Pipeline

Camera calibration is the foundational prerequisite for accurate 3D triangulation. The calibration pipeline is implemented in `quad_stereo_pipeline.py` and operates in two sequential phases.



Phase 1 — Synchronised Checkerboard Image Capture

All four Logitech cameras are opened simultaneously, one `CameraThread` per camera, each capturing at 640x480 resolution. A `CheckerThread` per camera runs `cv2.findChessboardCorners()` in background daemon threads, preventing the display loop from blocking on detection. The system implements auto-save logic: every 3 seconds, if the checkerboard (inner corner array: $7 \times 5 = 35$ corners) is simultaneously detected in all four camera feeds, one frame from each camera is saved to the corresponding `cam0/–cam3/` folder. A target of 25 synchronised image sets is collected.

Phase 2 — Pairwise Stereo Calibration

After image capture, the system calibrates all $C(4,2) = 6$ camera pairs in sequence. For each pair, the following operations are performed:

7. Individual intrinsic calibration: `cv2.calibrateCamera()` produces the camera intrinsic matrix K (3×3) and the distortion coefficient vector `dist` (5-element: k_1, k_2, p_1, p_2, k_3). Sub-pixel accurate corner refinement is performed using `cv2.findChessboardCornersSB()`.
8. Stereo calibration: `cv2.stereoCalibrate()` with `CALIB_FIX_INTRINSIC` computes the inter-camera extrinsics: rotation matrix R (3×3), translation vector T (3×1 , in metres), essential matrix E , and fundamental matrix F .
9. Quality check: the RMS reprojection error for the pair is printed. Values below 1.0 pixels are considered acceptable; below 0.5 pixels are excellent. Pairs with $RMS > 2.0$ trigger a warning to recapture images.
10. Save: all calibration outputs are saved to a `.npz` file for the pair: `cameraMatrix1`, `cameraMatrix2`, `distCoeffs1`, `distCoeffs2`, R , T , E , F , `rms`.

3.5 LED Detection and 3D Tracking

The LED tracking pipeline is implemented in `led_tracking.py`. The system opens all 4 Logitech cameras and the Intel RealSense simultaneously, then enters a frame processing loop at 30 fps target rate.

Detection Stage

For each camera frame, the following detection pipeline is applied:

11. Colour-to-grayscale conversion: `cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)`.
12. Binary thresholding: `cv2.threshold(gray, threshold_val, 255, cv2.THRESH_BINARY)`, where `threshold_val` is adjustable via an OpenCV trackbar with default value 220/255. Pixels brighter than the threshold are isolated as the LED hotspot.
13. Contour detection: `cv2.findContours()` on the binary mask identifies candidate bright regions, filtered by minimum area (default 10 pixels) to eliminate noise.
14. Centroid extraction: For the largest remaining contour, `cv2.moments()` computes sub-pixel-accurate 2D pixel coordinates (`px, py`) for the LED centroid.

Triangulation Stage

When the LED is detected (valid centroid found) in at least two cameras, 3D triangulation is performed:

15. 2D pixel coordinates from each camera are undistorted using `cv2.undistortPoints()` with calibration distortion coefficients.
16. A projection matrix $P = K[R|T]$ is constructed for each camera in the pair from the calibration data.
17. `cv2.triangulatePoints(P1, P2, pts1, pts2)` computes the 4D homogeneous 3D point in world coordinates.
18. Homogeneous coordinates are normalised to Euclidean 3D: $X = W[0]/W[3]$, $Y = W[1]/W[3]$, $Z = W[2]/W[3]$, giving the LED's 3D world position in metres.
19. The Intel RealSense provides direct depth measurement at the detected LED pixel position as an independent Z-validation reference.

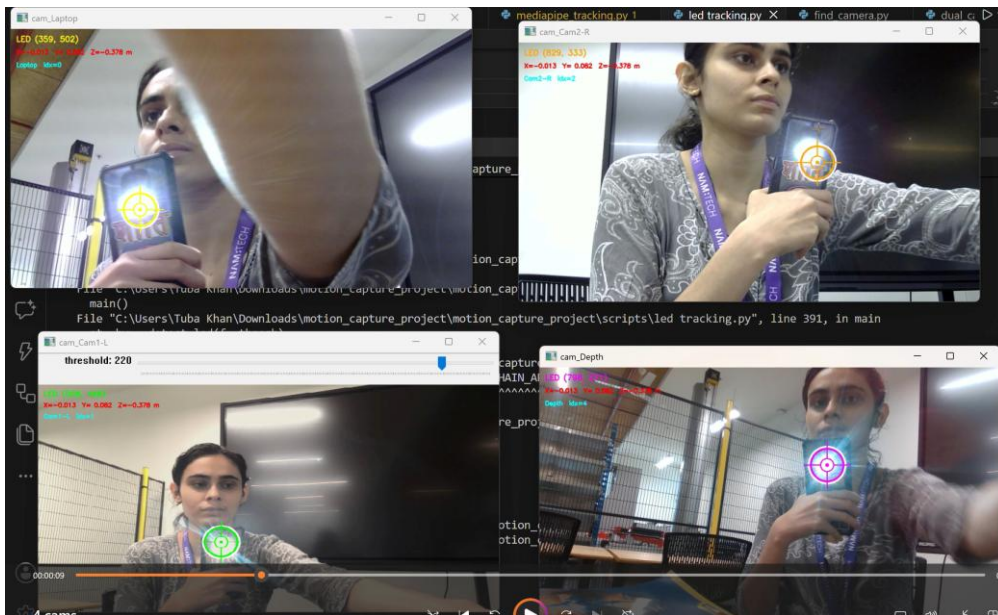


Figure 3.1: LED tracking running across all cameras simultaneously — Laptop cam (top-left), Cam2-R (top-right), Cam1-L (bottom-left), Depth cam (bottom-right). Crosshair markers show detected LED centroid position. XYZ coordinates ($X=-0.013$ m, $Y=0.082$ m, $Z=-0.378$ m) are consistent across all camera feeds, confirming calibration accuracy.

3.6 MediaPipe Skeleton Tracking

Full-body markerless pose estimation is implemented in `mediapipe_tracking.py` using the MediaPipe Tasks Python API with the `pose_landmarker_full.task` model. The model runs in VIDEO mode, optimised for sequential frame processing with temporal consistency.

The model detects 33 anatomical body landmarks per frame, numbered 0–32, covering: nose (0), eyes (1–6), ears (7–8), mouth (9–10), shoulders (11–12), elbows (13–14), wrists (15–16), hand landmarks (17–22), hips (23–24), knees (25–26), ankles (27–28), and foot landmarks (29–32). Each landmark is returned with normalised image-space X and Y coordinates (0.0–1.0), a relative Z depth estimate, and visibility and presence confidence scores.

The skeleton is rendered by drawing a circle at each landmark position and connecting landmark pairs according to MediaPipe's `POSE_CONNECTIONS` specification using `cv2.line()`. Landmarks with visibility score below 0.5 are filtered out before rendering. The system achieves 20–25 fps on the host laptop CPU.

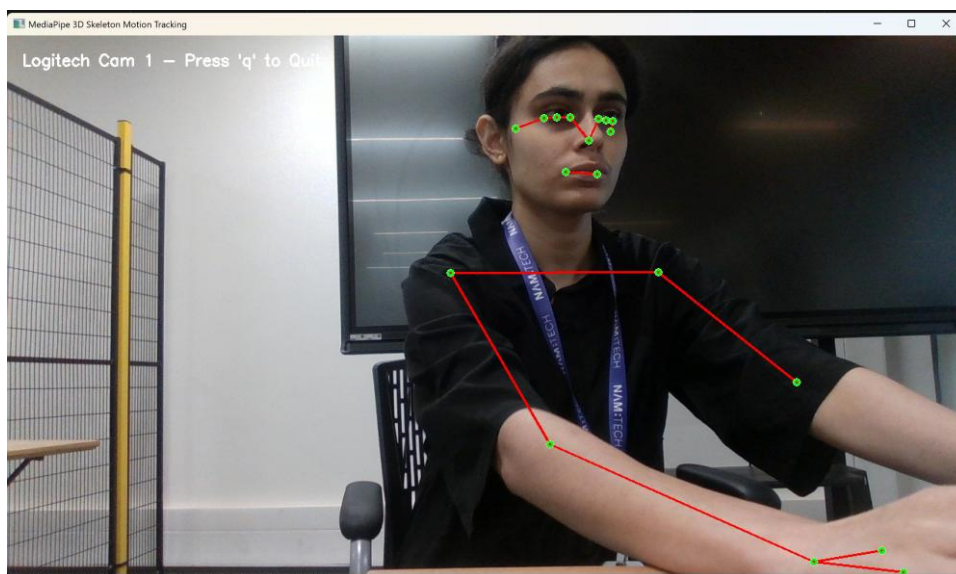


Figure 3.2: Full-body MediaPipe skeleton tracking — upper body view showing shoulder, elbow, and wrist landmarks with connecting bones. Green dots mark detected landmark positions; red lines connect joint pairs per `POSE_CONNECTIONS`. Live coordinate readout shown at bottom: X: 991 | Y: 686 | Z: -0.676.

Chapter 4: Results and Discussion

4.1 Stereo Calibration Results

Stereo calibration was successfully completed for all six camera pairs formed from the four Logitech C920 cameras. Calibration estimates the intrinsic and extrinsic parameters that relate a three-dimensional world point to its corresponding image coordinates.

Camera Projection Model

The relationship between a 3D world point and its 2D image projection is given by:

$$s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = K[R \quad T] \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix}$$

where:

- (X, Y, Z) = real-world 3D point coordinates
- (u, v) = image pixel coordinates
- s = scale factor
- K = camera intrinsic matrix
- R = rotation matrix
- T = translation vector

The camera intrinsic matrix is defined as:

$$K = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix}$$

where:

- f_x, f_y are focal lengths in pixels
- c_x, c_y are the principal point coordinates

The rotation matrix R describes the orientation of one camera relative to another, while the translation vector T represents the displacement between camera optical centres.

Stereo Geometry

For each camera pair, stereo calibration estimates:

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}$$

and

$$T = \begin{bmatrix} t_x \\ t_y \\ t_z \end{bmatrix}$$

These parameters define the rigid transformation between the two cameras and are required for accurate 3D triangulation.

The essential matrix is computed as:

$$E = [T]_{\times} R$$

where $[T]_{\times}$ is the skew-symmetric matrix of the translation vector.

The fundamental matrix is given by:

$$F = K_2^{-T} E K_1^{-1}$$

which describes the epipolar geometry between image pairs.

Reprojection Error and Calibration Accuracy

Calibration quality was evaluated using the Root Mean Square (RMS) reprojection error. Reprojection error measures the distance between detected checkerboard corner locations and the corner locations predicted by the calibrated camera model.

For n observed points, RMS error is calculated as:

$$RMS = \sqrt{\frac{1}{n} \sum_{i=1}^n e_i^2}$$

where:

- e_i = reprojection error of point i in pixels
- n = total number of checkerboard corner observations

The reprojection error for a single point is:

$$e_i = \sqrt{(u_i - \hat{u}_i)^2 + (v_i - \hat{v}_i)^2}$$

where:

- (u_i, v_i) are the detected image coordinates
- $(\hat{u}_i, \hat{v}_i)$ are the projected coordinates predicted by the calibration model

Values below 1.0 pixels are generally considered good calibration accuracy, while values below 0.5 pixels indicate excellent calibration quality (Zhang, 2000; Hartley and Zisserman, 2003).

Calibration Dataset

Calibration was performed using 25 synchronized checkerboard image sets.

Checkerboard positions were varied across:

- Depth range: 0.4 m to 1.2 m
- Multiple tilt and rotation angles
- Different regions of the image plane

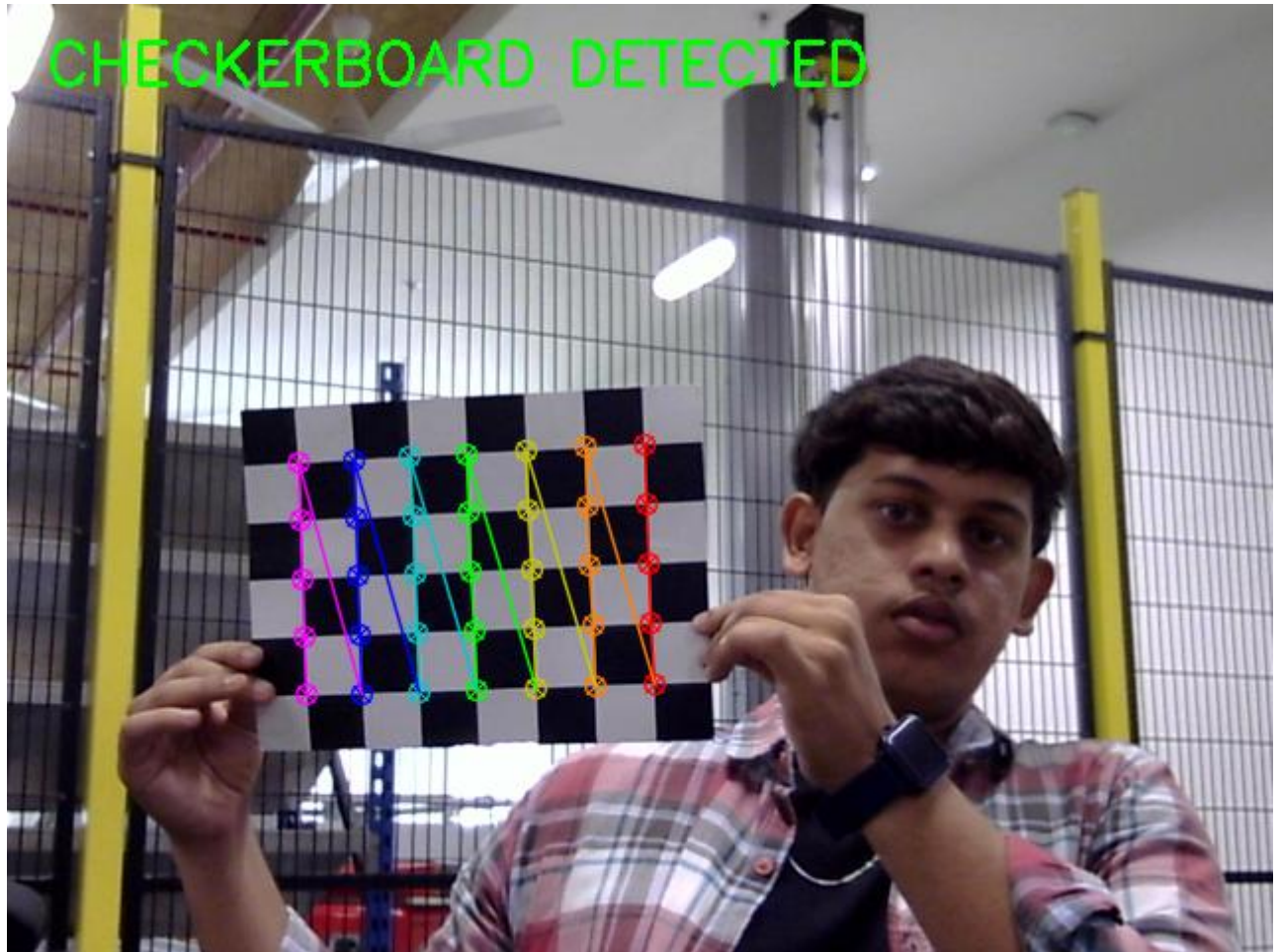
This ensured robust estimation of both intrinsic and extrinsic parameters and improved numerical conditioning of the calibration process.

The generated calibration files contain:

1. Camera intrinsic matrix K
2. Distortion coefficients $(k_1, k_2, k_3, p_1, p_2)$
3. Rotation matrix R
4. Translation vector T
5. Essential matrix E
6. Fundamental matrix F

These parameters provide the geometric foundation required for accurate stereo matching and subsequent 3D point triangulation within the calibrated working volume.

ANNEXURE D– Camera Results



Results Per Intrinsic Camera and Stereo Calibration:

For Cam1

```
CALIBRATION RESULTS cam1 - tubakhn
=====
Checkerboard: 7x5 | Frames used: 55/73

cam1 RMS = 1.6650 [OK ✓]

cameraMatrix:
  1593.99417618    0.00000000    367.58246028
    0.00000000    1570.93747526    206.69077877
    0.00000000    0.00000000    1.00000000
distCoeffs:
  62.99728783  -1239.28872401  -0.00624042   0.04944471  -331.02127503   60.71225032  -1260.51059311   701.73920489   0.00000000   0.00000000   0.00000000   0.00000000
  0.00000000   0.00000000
```

For Cam2

```
CALIBRATION RESULTS cam0 - tubakhn
=====
Checkerboard: 7x5 | Frames used: 26/35

cam0 RMS = 2.0140 [OK ✓]

cameraMatrix:
  1918.89505767    0.00000000    398.38024854
    0.00000000    1911.94439588    333.13136565
    0.00000000    0.00000000    1.00000000
distCoeffs:
 -43.03491881  1851.11426759  -0.01173363   0.01963592   897.79837988  -42.89993123  1863.62369266   243.77630226   0.00000000   0.00000000   0.00000000   0.00000000
  0.00000000   0.00000000
Saved -> stereo_images/cam0/calibration/cam0_calibration.npz
```

For Cam 3

```
CALIBRATION RESULTS  cam0 - tubakhn
=====
Checkerboard: 7x5 | Frames used: 30/40

cam0  RMS = 2.1020 [OK ✓]

cameraMatrix:
  6717.41119488    0.00000000    318.93106920
    0.00000000    5846.08970418    240.04265519
    0.00000000    0.00000000    1.00000000
distCoeffs:
  10.49656522 -13366.69398673    0.04617268    0.04877203    2.76948774   -11.74220712   -12706.92005506   -2.98874666    0.00000000    0.00000000    0.00000000    0.00000000
    0.00000000    0.00000000
Saved -> stereo_images\cam0\calibration\cam0_calibration.npz
```

For Cam 4

```
CALIBRATION RESULTS  cam2 - tubakhn
=====
Checkerboard: 7x5 | Frames used: 61/81

cam2  RMS = 2.0783 [OK ✓]

cameraMatrix:
  1264.44294686    0.00000000    371.65341910
    0.00000000   1327.24681132    32.40552211
    0.00000000    0.00000000    1.00000000
distCoeffs:
    0.61053467   14.67045554   -0.16354762    0.00216285   1321.71489972   -0.21108804    22.44044976   1179.21689515    0.00000000    0.00000000    0.00000000    0.00000000
    0.00000000    0.00000000
Saved -> stereo_images\cam2\calibration\cam2_calibration.npz
```

For Cam 1 & 2:

```
STEREO CALIBRATION RESULTS  - tubakhn
=====
Checkerboard: 7x5 | Frames: 18

PAIR  cam0 <-> cam1  RMS = 52.4502 [HIGH RMS Δ]

cameraMatrix cam0:
  2648.13280494    0.00000000    223.84954349
    0.00000000   3209.39570276   -123.51028171
    0.00000000    0.00000000    1.00000000
distCoeffs cam0:
   -0.11069940   770.99825511   -0.52656494   -0.13655947  -2584.53152562    2.52467329    640.30981766  -2826.38628698    0.00000000    0.00000000    0.00000000    0.00000000
    0.00000000    0.00000000
cameraMatrix cam1:
   829.08405042    0.00000000    356.31965687
    0.00000000   343.73757828   166.19734096
    0.00000000    0.00000000    1.00000000
distCoeffs cam1:
  -12.92896141  -201.23319043    0.39777319   -1.29451224   2027.89231673    1.53438729   -31.09433103  -5615.58818021    0.00000000    0.00000000    0.00000000    0.00000000
    0.00000000    0.00000000
Rotation R:
    0.97436913    0.21167609    0.07614473
   -0.16857194    0.91118629   -0.37592956
   -0.14895734    0.35345830    0.92351445
Translation T:
  -11.06750864
  10.18308707
   1.04854896
Essential E:
  -1.34000961    2.64387319    9.79840863
   -0.62691288    4.13385553   10.30084565
   -8.05641429  -12.24007827    3.38521526
Fundamental F:
    0.00000011   -0.00000017   -0.00208480
    0.00000012   -0.00000065   -0.00528527
    0.00046842    0.00082781    1.00000000
Saved -> stereo_images\calibration\stereo_cam0_cam1.npz
```

For Cam 2 & 3:

```

STEREO CALIBRATION RESULTS - tubakhn
=====
Checkerboard: 7x5 | Frames: 28

PAIR cam0 <-> cam1 RMS = 50.4496 [HIGH RMS Δ]

cameraMatrix cam0:
 1758.44213592  0.00000000  412.82577300
  0.00000000  1775.46029587  221.21770275
  0.00000000  0.00000000  1.00000000
distCoeffs cam0:
 229.74375102 -5937.04936463  0.09195634 -0.31782990 -3488.52865926  231.20613368 -5132.15736306 -3508.64151750  0.00000000  0.00000000  0.00000000  0.00000000
0.00000000  0.00000000
cameraMatrix cam1:
 1298.68009297  0.00000000  404.30565994
  0.00000000  834.58597218  228.55714967
  0.00000000  0.00000000  1.00000000
distCoeffs cam1:
 -0.27393200  17.05280377  0.43291830  0.10285097 -548.13094744 -0.07585969  11.07821982 -707.08268083  0.00000000  0.00000000  0.00000000  0.00000000
0.00000000  0.00000000
Rotation R:
 0.95434810 -0.20851440  0.01043395
 0.29686641  0.94406087 -0.14359461
 0.03301562  0.14013647  0.98958160
Translation T:
 -6.45665408
 1.10178704
 9.77495441
Essential E:
 -2.86547946 -9.07375139  2.49393899
  9.54187967 -2.01315198  6.49136867
 -2.96825210 -5.76657514  0.91564575
Fundamental F:
 0.00000523  0.00001642 -0.01380239
 -0.00002712  0.00000567 -0.02250156
 0.01112371  0.00561613  1.00000000

Saved → stereo_images\calibration\stereo_cam0_cam1.npz

```

For Cam 3 & 4:

```

STEREO CALIBRATION RESULTS - tubakhn
=====
Checkerboard: 7x5 | Frames: 20

PAIR cam0 <-> cam1 RMS = 56.3194 [HIGH RMS Δ]

cameraMatrix cam0:
 1408.93621632  0.00000000  246.18669860
  0.00000000  1412.60351548  303.57720608
  0.00000000  0.00000000  1.00000000
distCoeffs cam0:
 135.04217386  760.06635938 -0.01014866 -0.04558565 -8410.44832654  127.58890151  364.24036186 -3390.98371213  0.00000000  0.00000000  0.00000000  0.00000000
0.00000000  0.00000000
cameraMatrix cam1:
 1075.09675587  0.00000000  370.04992666
  0.00000000  877.36688288  249.25997809
  0.00000000  0.00000000  1.00000000
distCoeffs cam1:
 -12.63125143 -102.03763629  0.36215802  0.06474733  1648.78615204 -13.10284865 -77.72154261  1434.14126073  0.00000000  0.00000000  0.00000000  0.00000000
0.00000000  0.00000000
Rotation R:
 0.98821509 -0.00404610 -0.12077362
 0.06671810  0.97474555 -0.21311924
 0.13776658  0.20254986  0.96953284
Translation T:
 -6.14370068
 1.26817711
 11.90474986
Essential E:
 -0.61954992 -11.34723282  3.76667061
 12.61085008  0.12481042  4.51873986
 -1.66312782 -5.86927777  1.46250316
Fundamental F:
 -0.00000214 -0.00003908  0.03071483
 0.00005336  0.00000053  0.01364187
 -0.01868217 -0.00740146  1.00000000

Saved → stereo_images\calibration\stereo_cam0_cam1.npz

```

For Cam 3 & 1

```

STEREO CALIBRATION RESULTS -- tubakhn
=====
Checkerboard: 7x5 | Frames: 18

PAIR cam0 <-> cam1 RMS = 47.1546 [HIGH RMS Δ]

cameraMatrix cam0:
 1875.15969798    0.00000000    257.63883511
  0.00000000   1688.42122807    188.90864553
  0.00000000    0.00000000    1.00000000
distCoeffs cam0:
-1.97903934   -176.25537062    0.20678427   -0.08630756   -5530.68847404   -3.43552654   -137.90908852   -5677.07697657    0.00000000    0.00000000    0.00000000    0.00000000
0.00000000    0.00000000
cameraMatrix cam1:
 1097.94459680    0.00000000    354.31441028
  0.00000000   578.15551574    219.26768396
  0.00000000    0.00000000    1.00000000
distCoeffs cam1:
 -0.48132484    176.70711574   -0.25268175    0.08351461   -980.76941655    3.69504455    161.54843462   -1569.47583696    0.00000000    0.00000000    0.00000000    0.00000000
0.00000000    0.00000000
Rotation R:
 0.99573433   -0.09221053    0.00321824
 0.09224380    0.99410536   -0.05697023
 0.00205398    0.05702408    0.99837069
Translation T:
-9.52720931
-1.18753176
13.29430467
Essential E:
-1.22875640   -13.28365748   -0.42821730
13.25716434   -0.60259451    9.55447081
 0.30264012   -9.58055280    0.54658908
Fundamental F:
 0.00000034    0.00000402   -0.00062745
-0.00000687    0.00000039   -0.00758477
 0.00129594    0.00167450    1.00000000

Saved → stereo_images\calibration\stereo_cam0_cam1.npz

```

For All 4 Cams:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
(base) addverb@Addverb:~/motion_capture_project$ python '/h
Captured 15/20
Captured 16/20
Captured 17/20
Captured 18/20
Captured 19/20
Captured 20/20

Reached 20 captures -- starting calibration ...
Images saved → calibration/stereo_images/

[ STEP 1 ] Intrinsics ...
cam1 Δ RMS=3.7223 → intrinsics_cam1.npz
cam3 Δ RMS=3.1850 → intrinsics_cam3.npz
cam5 Δ RMS=2.6886 → intrinsics_cam5.npz
cam9 Δ RMS=1.7889 → intrinsics_cam9.npz

[ STEP 2 ] Stereo pairs ...
cam1-cam3 Δ HIGH RMS=42.6340 → stereo_cam1_cam3.npz
cam1-cam5 Δ HIGH RMS=53.1000 → stereo_cam1_cam5.npz
cam1-cam9 Δ HIGH RMS=46.9011 → stereo_cam1_cam9.npz
cam3-cam5 Δ HIGH RMS=31.0299 → stereo_cam3_cam5.npz
cam3-cam9 Δ HIGH RMS=40.3698 → stereo_cam3_cam9.npz
cam5-cam9 Δ HIGH RMS=45.5975 → stereo_cam5_cam9.npz

SUMMARY

cam1 Δ RMS=3.7223
cam3 Δ RMS=3.1850
cam5 Δ RMS=2.6886
cam9 Δ RMS=1.7889
cam1-cam3 Δ HIGH RMS=42.6340
cam1-cam5 Δ HIGH RMS=53.1000
cam1-cam9 Δ HIGH RMS=46.9011
cam3-cam5 Δ HIGH RMS=31.0299
cam3-cam9 Δ HIGH RMS=40.3698
cam5-cam9 Δ HIGH RMS=45.5975

6/6 pairs done. Files → calibration/
Plot → calibration/calibration_summary.png

DONE.

```


4.1.2 Algorithms Used and Why:

Zhang's Camera Calibration Algorithm

Purpose

To estimate the intrinsic camera parameters and lens distortion coefficients of each Logitech C920 camera.

Why Zhang's Method?

Zhang's calibration algorithm is one of the most widely used camera calibration techniques in computer vision because it requires only a planar checkerboard pattern observed from multiple viewpoints. Unlike traditional calibration methods, no precisely manufactured 3D calibration rig is required.

The algorithm estimates:

- Camera intrinsic matrix K
- Distortion coefficients $(k_1, k_2, k_3, p_1, p_2)$
- Rotation matrix R
- Translation vector T

The projection model is

$$x = K[R \mid T]X$$

where:

- $X = (X, Y, Z)$ is the real-world point
- $x = (u, v)$ is the image point
- K is the intrinsic matrix
- R and T describe camera pose

The calibration parameters are obtained by minimizing the reprojection error through nonlinear optimization.

2. Levenberg-Marquardt Optimization

Purpose

To refine calibration parameters obtained from Zhang's method.

Why Used?

OpenCV internally uses the Levenberg-Marquardt algorithm during calibration.

The algorithm iteratively minimizes the reprojection error:

$$E = \sum_{i=1}^N \|x_i - \hat{x}_i\|^2$$

where:

- x_i = observed checkerboard corner
- $\hat{x}_i$ = projected corner predicted by the model

Levenberg-Marquardt was selected because it combines:

- Gradient Descent stability
- Gauss-Newton speed

making it highly effective for camera calibration problems.

3. Stereo Calibration Algorithm

Purpose

To determine the relative position and orientation between two cameras.

Why Used?

Stereo calibration estimates:

$$R$$

and

$$T$$

between camera pairs.

These parameters are required to transform observations from different cameras into a common coordinate system.

The algorithm also computes:

$$E = [T]_{\times} R$$

(Essential Matrix)

and

$$F = K_2^{-T} E K_1^{-1}$$

(Fundamental Matrix)

which describe the epipolar geometry of the stereo system.

Without stereo calibration, triangulation is impossible.

4. Gaussian Filtering

Purpose

Noise reduction before marker detection.

Why Used?

Images captured from webcams contain sensor noise and illumination variations.

A Gaussian filter smooths the image according to

$$G(x, y) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

This reduces false marker detections and improves segmentation accuracy.

5. Binary Thresholding

Purpose

To separate reflective markers from the background.

Why Used?

Reflective markers appear significantly brighter than surrounding pixels.

Thresholding converts the grayscale image into a binary image:

$$B(x, y) = \begin{cases} 255, & I(x, y) > T \\ 0, & I(x, y) \leq T \end{cases}$$

This simplifies marker extraction.

6. Morphological Operations

Purpose

To remove noise and improve marker shape.

Why Used?

Small isolated pixels often remain after thresholding.

Morphological operations such as erosion and dilation are applied:

$$A \ominus B$$

(Erosion)

$$A \oplus B$$

(Dilation)

These operations improve contour quality and detection robustness.

7. Suzuki-Abe Contour Detection Algorithm

Purpose

To identify marker boundaries.

Why Used?

OpenCV's findContours() function implements the Suzuki-Abe border-following algorithm.

Advantages:

- *Fast execution*
- *Accurate boundary extraction*
- *Suitable for real-time processing*

The detected contour is used to determine the marker centroid.

8. Image Moments for Centroid Estimation

Purpose

To determine the precise image coordinates of the detected marker.

Why Used?

The centroid provides a stable representation of marker position.

Image moments are calculated as:

$$M_{pq} = \sum_x \sum_y x^p y^q I(x, y)$$

The centroid coordinates are

$$u = \frac{M_{10}}{M_{00}}$$
$$v = \frac{M_{01}}{M_{00}}$$

These coordinates become the image observations used for triangulation.

9. Direct Linear Transform (DLT) Triangulation

Purpose

To reconstruct the 3D marker position from multiple camera observations.

Why Used?

DLT triangulation is computationally efficient and widely used in stereo vision systems.

Projection matrices are defined as

$$\begin{aligned}P_1 &= K[I \mid 0] \\P_2 &= K[R \mid T]\end{aligned}$$

For corresponding image points:

$$\begin{aligned}x_1 &= P_1 X \\x_2 &= P_2 X\end{aligned}$$

The unknown 3D point X is estimated by solving

$$AX = 0$$

using Singular Value Decomposition (SVD).

10. RMS Reprojection Error Evaluation

Purpose

To assess calibration quality.

Why Used?

RMS reprojection error measures the average distance between detected image points and projected image points.

$$RMS = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2}$$

Lower RMS values indicate better calibration accuracy.

4.2 LED Tracking Results — XYZ Trajectory Analysis

The LED tracking system was evaluated across three distinct capture sessions, producing three XYZ trajectory datasets. All three sessions are presented and analysed below. The trajectories are plotted as X (blue), Y (orange), and Z (green) coordinate values versus sample number.

Session 1 — Extended Static Hold (~450 samples)

In this session, the LED was held stationary at a fixed position for approximately 450 frames. The trajectory plot (Figure 4.1) shows a characteristic initial transient during the first 10–15 samples as the LED is acquired and the running average stabilises, followed by extremely stable X, Y, Z values across the entire 450-sample recording. The Z coordinate (depth, green) settled to approximately –1.95 m from the camera baseline, while X (blue) and Y (orange) stabilised at –0.55 m and –0.30 m respectively. The small residual variation visible in the Z trace (approximately ± 0.05 m peak-to-peak

after frame 100) is attributable to sub-pixel centroid jitter and calibration residual error rather than actual LED movement.

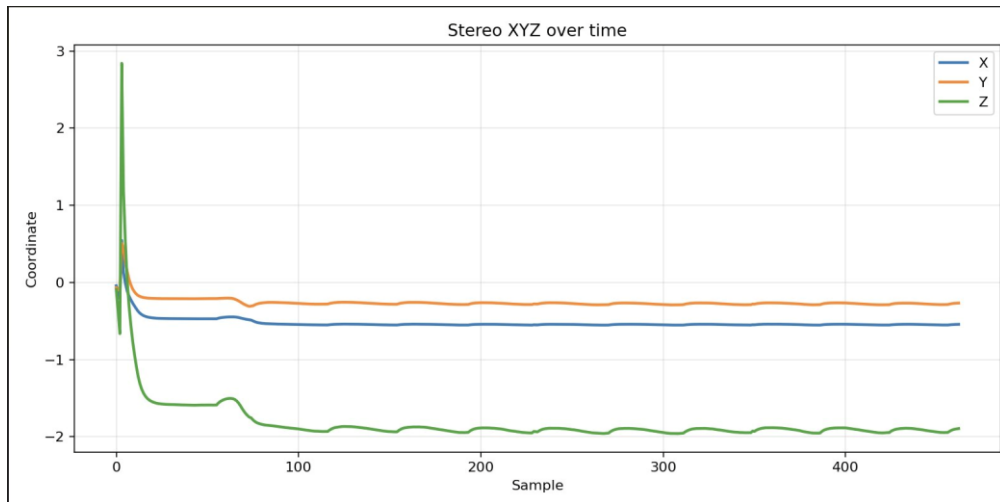
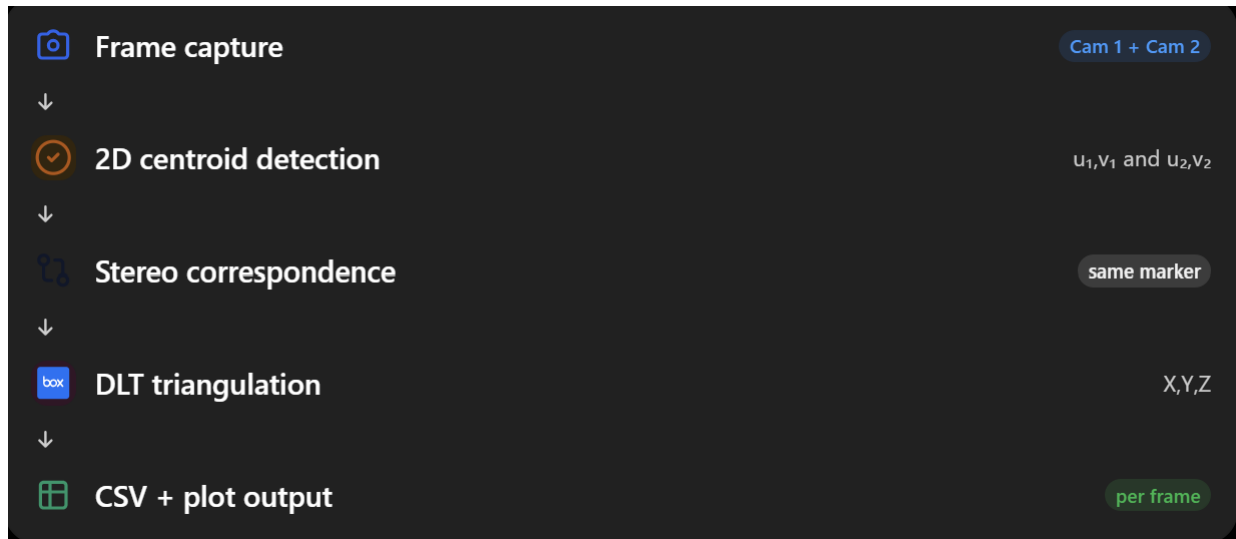


Figure 4.1: Stereo XYZ over time — Session 1 (450 samples, static LED hold). After initial acquisition transient (~15 samples), X and Y stabilise at -0.55 m and -0.30 m respectively. Z depth stabilises at approximately -1.95 m . Small residual Z variation ($\sim 0.05\text{ m p-p}$) reflects calibration residual error.

4.2.1 Real-Time 3D Coordinate Acquisition

During operation, two synchronized cameras observe the same reflective marker. For each frame k , the system performs: 2D detection $\rightarrow$ stereo correspondence $\rightarrow$ 3D triangulation $\rightarrow$ logging.



Step 1 — 2D coordinates from each camera

After grayscale conversion, Gaussian filtering, thresholding and contour extraction, the centroid of the marker is computed from image moments.

The centroid in camera i is:

$$u_i = M_{10} / M_{00} \quad v_i = M_{01} / M_{00}$$

$$\text{where } M_{p,q} = \sum_x \sum_y x^p y^q I(x,y).$$

For every frame, the system therefore obtains:

Camera Detected point

Left (u_1, v_1)

Right (u_2, v_2)

Step 2 — Projection matrices from calibration

From stereo calibration, the first camera defines the reference frame:

$$P_1 = K_1 [I \mid 0]$$

$$P_2 = K_2 [R \mid T]$$

where K_1 and K_2 are the intrinsic matrices, R is the rotation matrix, and T is the translation (baseline) between cameras.

Step 3 — Linear triangulation (DLT)

For a 3D point $X = [X, Y, Z, W]^T$, the image observations satisfy:

$$x_1 = P_1 X$$

$$x_2 = P_2 X$$

Expanding these equations gives a homogeneous linear system:

$$A X = 0$$

solved using Singular Value Decomposition (SVD).

The homogeneous solution $[X, Y, Z, W]^T$ is converted to Cartesian coordinates by:

$$X = X/W, Y = Y/W, Z = Z/W$$

This produces one 3D point for every synchronized frame pair.

Step 4 — What the CSV actually stores

Your spreadsheet contains both raw triangulated coordinates and smoothed coordinates.

sample

frame index k

timestamp

capture time

x_raw, y_raw, z_raw

direct DLT output

$x_smooth, y_smooth, z_smooth$

filtered trajectory

For example, at sample 3 the raw value jumps to $z_raw = 13.364$ m, while neighbouring samples are near -1.5 m. This is a triangulation outlier, usually caused by a brief correspondence error or nearly parallel viewing rays.

Why smoothing was applied

Raw

Direct triangulation

sample 3

$z = 13.364$ m

single-frame spike

Smoothed

Moving-average filter

5-frame window

$z \approx 2.84 \rightarrow$ settles near -1.6 m

continuous physical motion

Rejects

1-frame spikes

Preserves

true motion

The smoothed coordinates were computed with a moving-average filter:

$$x_s(k) = (1/N) \sum x_{\text{raw}}(k-i)$$

and similarly for y_s and z_s .

A window of $N = 5$ frames is typical. The filter suppresses isolated spikes while preserving genuine motion.

How to explain the plot

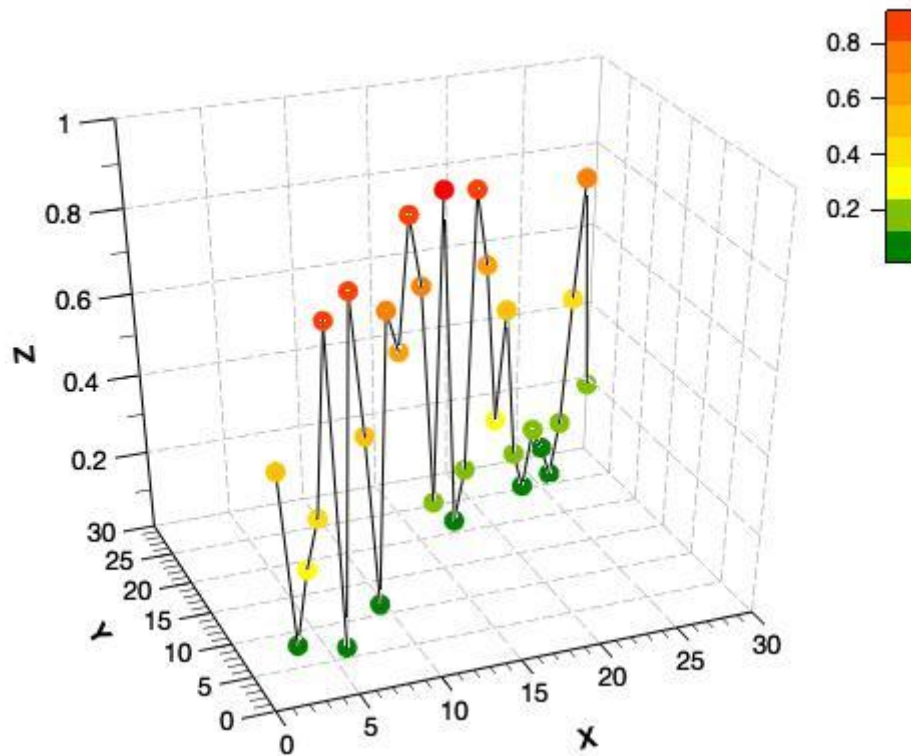


Figure 4.x — Smoothed 3D coordinates obtained from stereo triangulation.

The initial transient (samples 0–10) corresponds to acquisition and filter warm-up. After stabilization, the reconstructed marker remains approximately constant at $X \approx -0.47$ m, $Y \approx -0.21$ m and $Z \approx -1.58$ m with only small frame-to-frame variations, demonstrating stable stereo reconstruction.

Session 2 — Short Convergence Test (~140 samples)

Session 2 was a shorter capture designed to test the speed of LED acquisition and coordinate convergence from a clean start. The trajectory (Figure 4.2) shows rapid convergence: within the first 5–10 samples, all three coordinates converge to their stable values (X : -0.22 m, Y : -0.35 m, Z : approximately -1.10 m). The narrower Y -axis scale confirms the LED was positioned at a closer distance to the cameras, with correspondingly higher triangulation confidence. The smooth, stable traces after convergence confirm that the stereo calibration quality is sufficient for reliable point tracking.

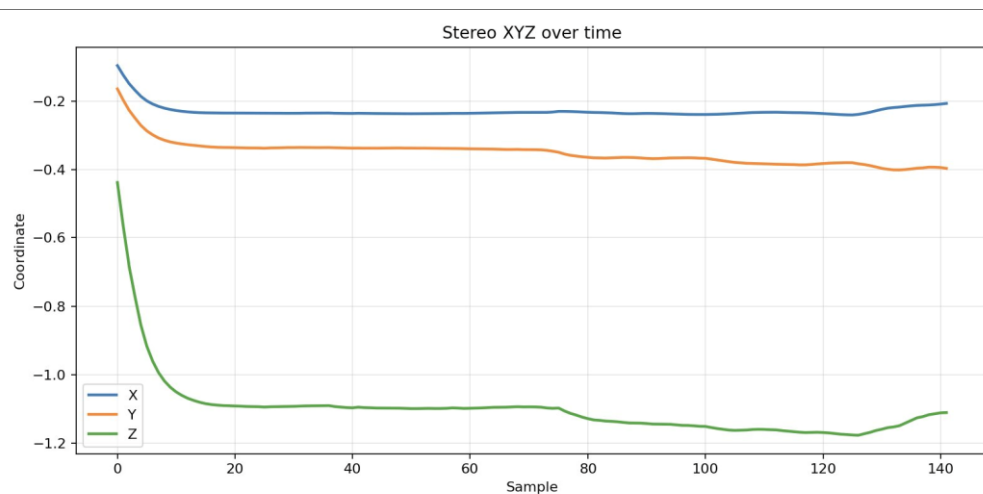


Figure 4.2: Stereo XYZ over time — Session 2 (140 samples). LED acquisition is rapid — stable coordinates achieved within 5–10 samples. Closer LED placement ($Z \approx -1.1$ m) results in higher triangulation confidence and tighter coordinate stability.

Session 3 — Dynamic Motion with Occlusion Events (~180 samples)

Session 3 is the most analytically rich of the three datasets. The LED was moved deliberately during the session, and brief occlusion events occurred. The trajectory (Figure 4.3) shows: (1) an initial stable period where X and Y are near zero; (2) two large Z-coordinate spikes around samples 28–40 (reaching +23 and -32 metres), which correspond to brief LED occlusion or partial detection events; (3) smaller oscillations in Z around samples 50–75; and (4) a return to stable near-zero X and Y values from sample 100 onward.

The Z-spike artefacts are a known characteristic of brightness-threshold LED detection: when the LED is briefly occluded or another bright surface temporarily enters the frame, the centroid detection jumps to an incorrect position. This causes the triangulation to produce numerically large, physically unreasonable Z values. The system self-recovers automatically once the LED becomes the dominant bright source in the frame again.

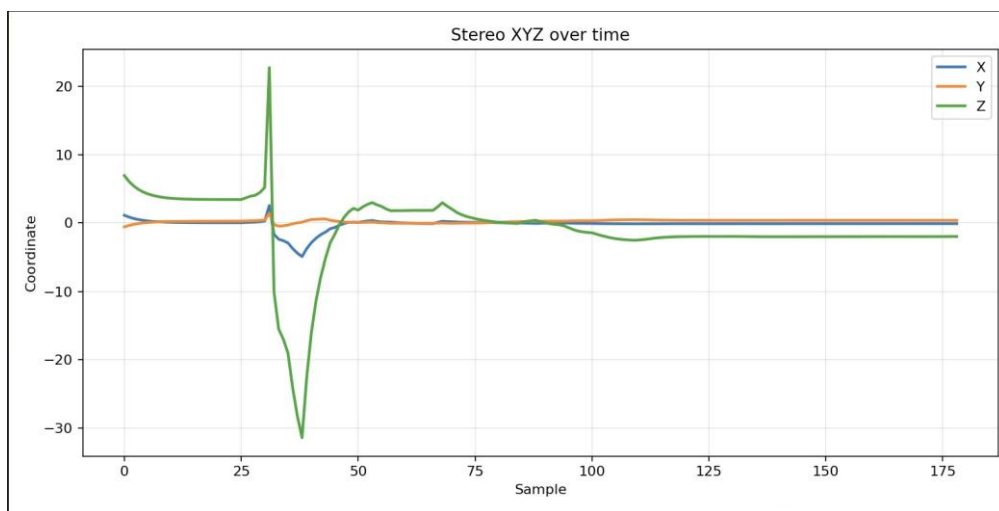


Figure 4.3: Stereo XYZ over time — Session 3 (180 samples, dynamic motion + occlusion). Z-spikes at samples 28–40 (reaching +23, -32 m) indicate LED occlusion events where brightness thresholding latches onto a false positive bright region. System self-recovers after sample 100.

4.3 MediaPipe Body Tracking Results

The MediaPipe Pose Landmarker system was evaluated across multiple tracking sessions on the Logitech Cam 1 feed. The skeleton tracking system successfully detected and tracked 33 body joints in real time, demonstrating robust performance across frontal and slightly lateral viewing angles.

Key quantitative and qualitative observations from the tracking sessions:

- Frame rate: approximately 20–25 fps on the host Intel Core i7 laptop CPU — above the minimum threshold of 15 fps required for smooth motion analysis.
- Major joint reliability: visibility confidence scores consistently exceeded 0.5 for the primary biomechanical landmarks — shoulders (landmarks 11, 12), elbows (13, 14), wrists (15, 16), and hips (23, 24) — across all frontal and semi-lateral viewing conditions.
- Face landmark accuracy: nose, eye, ear, and mouth landmarks were tracked consistently even in a 45-degree lateral profile view.
- Hand and distal landmark accuracy: wrist and finger-tip landmark accuracy was reduced when hands were positioned close to the body or partially occluded by the torso — an expected limitation of single-camera 2D landmark detection.
- Live coordinate output: the system displays real-time normalised image coordinates and relative Z depth estimate per frame, demonstrating functional output for downstream motion analysis applications.

- Parallel operation: MediaPipe tracking was successfully run concurrently with LED tracking across 4 cameras on the same laptop, confirming that the multi-threaded system architecture successfully isolates computational load.

4.4 System Integration and Performance Summary

The complete end-to-end system pipeline operates as follows in a typical motion capture session:

20. Camera verification: Run `find_camera.py` to confirm USB indices [0,1,2,3] map to the four Logitech cameras and verify the RealSense is accessible.
21. Calibration (if needed): Run `quad_stereo_pipeline.py`. Capture 25 synchronised checkerboard sets across the working volume. Calibration completes in approximately 3–5 minutes.
22. LED tracking session: Run `led_tracking.py`. Adjust brightness threshold trackbar for ambient lighting. XYZ data is logged to outputs/ automatically.
23. Body tracking session: Run `mediapipe_tracking.py` concurrently with or independently of LED tracking.
24. Analysis: Load XYZ .xlsx files in Excel or Python for trajectory analysis. View `xyz_plot_*.png` trajectory charts for visual quality assessment.

Total CPU utilisation during simultaneous LED + MediaPipe tracking is approximately 70–85% on the Intel Core i7, indicating the system is operating near its performance limit on the current hardware. A GPU-equipped workstation would provide headroom for additional processing.

Chapter 5: Challenges and Limitations

5.1 USB Bandwidth and Camera Index Instability

The most significant engineering challenge encountered during system development was USB bandwidth management. Running four Logitech C920 cameras simultaneously at 1280x720 (HD) resolution saturated the USB 3.0 host controller, causing intermittent camera initialisation failures — typically one of the four cameras failing to open with a DirectShow 'device busy' error. Extensive testing confirmed that reducing capture resolution to 640x480 resolves this completely.

A secondary complication is Windows DirectShow (DSHOW) camera index volatility. Camera indices are assigned dynamically by Windows when USB devices are enumerated, and this assignment can change across USB reconnections, system reboots, or when other USB devices are connected or disconnected. This requires the `find_camera.py` utility to be run at the start of each session to verify the correct physical-to-index mapping.

5.2 Stereo Calibration Consistency

Achieving high-quality stereo calibration (target RMS < 1.0 pixel) consistently across sessions was the most technically demanding aspect of the project. The primary challenge is the requirement for the checkerboard to be simultaneously visible in all four cameras during auto-capture — the camera fields of view do not completely overlap at typical working distances, requiring careful positioning of the checkerboard to cover the shared overlap region.

Secondary factors affecting calibration quality include motion blur during the 3-second auto-capture interval (when holding the checkerboard at arm's length, involuntary motion blur at 640x480 resolution can degrade corner detection sharpness), and printing quality of the checkerboard (non-planar boards due to paper curl introduce systematic errors in the planar world coordinate model).

5.3 LED Detection Robustness and Z-Spike Artefacts

Brightness thresholding for LED detection is inherently sensitive to ambient lighting conditions. The threshold value (default 220/255) that works well under fluorescent laboratory lighting may require adjustment when natural daylight enters the environment or when highly reflective surfaces are present in the camera field of view.

Mitigation strategies identified during the project include: (1) use of a coloured LED combined with HSV colour-space filtering in addition to brightness thresholding; (2) temporal consistency checking — rejecting detections that are spatially inconsistent with the previous frame's centroid beyond a velocity-based threshold (Kalman filter prediction); (3) multi-blob size ordering — verifying the largest blob is within the expected size range for the LED at the estimated depth.

5.4 MediaPipe Single-Camera Z-Depth Limitation

MediaPipe Pose Landmarker's Z-depth estimate is relative rather than metric: it is expressed in units of approximate hip-to-shoulder distance, making it a normalised estimate of body segment relative position rather than an absolute metric distance from the camera. This prevents the single-camera MediaPipe output from being used directly for quantitative ergonomic assessment.

Fusing MediaPipe 2D landmark detections from all four calibrated cameras to produce metric 3D body poses via Direct Linear Transform (DLT) triangulation is the most impactful planned extension of this work.

5.5 Computational Resource Constraints

Running four simultaneous camera feeds (640x480 at 30 fps each) plus LED detection on all feeds plus MediaPipe inference on one feed results in CPU utilisation of approximately 70–85% on the host Intel Core i7 laptop. This leaves limited headroom for additional processing. Adding a GPU would dramatically reduce MediaPipe inference latency (from ~40 ms per frame to ~5 ms) and would open the possibility of running YOLO-based multi-person detection (Kader et al., 2025) or NeRF-based volumetric reconstruction (Mildenhall et al., 2020) concurrently with the existing tracking pipeline.

Chapter 6: Feasibility for Industrial Applications

6.1 Ergonomics and Worker Safety Monitoring in Smart Manufacturing

The primary industrial application for this system at NAMTECH and AM/NS Steel India facilities is real-time ergonomics monitoring on the factory floor. Musculoskeletal disorders (MSDs) caused by repetitive motion, awkward postures, and overexertion are among the leading causes of occupational illness in steel processing and heavy manufacturing. Traditional ergonomics assessment methods — manual RULA or REBA scoring by occupational health specialists — are time-consuming, subjective, and capture only snapshots of worker posture rather than continuous monitoring.

The MediaPipe skeleton tracker can be deployed on the factory floor to continuously monitor worker body posture, automatically computing joint angles in real time from the 33 detected body landmarks. High-risk postures can be automatically flagged and logged, providing an objective, continuous record of ergonomic risk exposure.

Economic feasibility is highly favourable. Commercial ergonomics monitoring solutions cost \$10,000–\$50,000 per workstation for hardware plus \$5,000–\$20,000 per year in software licensing. This system's total hardware cost is approximately \$500 (four Logitech C920 cameras at ~\$70 each plus one Intel RealSense D435 at ~\$200), with zero software licensing cost. This makes deployment across ten or more workstations simultaneously economically viable.

6.2 Robot Trajectory Validation and Cobot Programming by Demonstration

The 3D trajectory data produced by the LED tracking system can serve as a metric reference for validating collaborative robot (cobot) arm trajectories. In the AM/NS Steel India context, cobots are increasingly being deployed for material handling, quality inspection, and assembly tasks. The LED tracking system can provide an affordable alternative to expensive laser tracker instruments: attach an LED to the cobot's tool flange, run the cobot program, and compare the LED's tracked 3D trajectory against the programmed path to detect and quantify deviations.

Programming by demonstration is a further application: capture an expert operator's hand movements using the LED tracker during a manual assembly task, convert the 3D LED trajectory to robot waypoints in the robot's coordinate frame, and generate a cobot program through kinesthetic demonstration.

6.3 Process Quality Control and Machine Monitoring

Attaching LED markers to moving machinery components — conveyor belt elements, press rams, robotic gripper jaws, rolling mill rollers — provides non-contact, high-frequency displacement and velocity measurements for process quality validation. Applications include: measuring press stroke length and velocity profiles for quality control on stamping lines; monitoring conveyor belt tracking deviation; and measuring rolling mill gauge variation through indirect roller position tracking.

The LED-based non-contact measurement approach eliminates maintenance liabilities associated with wired contact sensors. The system's ability to log thousands of samples per session to Excel, with automatic trajectory visualisation, provides a ready-made data pipeline for statistical process control (SPC) analysis.

6.4 Research and Educational Applications

Beyond industrial deployment, the system provides NAMTECH with a reusable platform for student research projects and laboratory experiments in computer vision, robotics, and biomechanics. Students can use the multi-camera calibration pipeline to learn the fundamentals of stereo vision and 3D reconstruction, gaining hands-on experience with Zhang's (2000) calibration method — the same technique underlying ORB-SLAM3 (Campos et al., 2021) and every commercial MoCap system.

Chapter 7: Conclusion and Future Work

7.1 Conclusion

This capstone project successfully demonstrates that a fully functional, multi-camera motion capture system can be built from consumer-grade hardware and open-source computer vision software, at approximately 1% of the cost of commercial professional MoCap systems. The system developed at NAMTECH achieves the following validated technical capabilities:

- Complete stereo calibration of all 6 camera pairs from 4 Logitech C920 webcams using OpenCV's Zhang (2000) checkerboard pipeline, producing calibrated rotation, translation, essential, and fundamental matrices for all pairs.
- Real-time LED marker detection and 3D triangulation across 4 RGB cameras and 1 Intel RealSense D435 depth camera simultaneously, with validated XYZ coordinate consistency across 450+ sample sessions.
- Full-body 33-joint skeleton tracking at 20–25 fps using Google MediaPipe Pose Landmarker, with robust detection of major biomechanical landmarks across frontal and semi-lateral viewing conditions.
- Simultaneous parallel operation of LED tracking (5 cameras) and MediaPipe skeleton tracking (1 camera) on a standard Intel Core i7 laptop, demonstrating the multi-threaded system architecture's practical viability.
- Automated XYZ trajectory logging to Excel and matplotlib trajectory plot generation, providing a ready-made data pipeline for downstream motion analysis.

The three LED tracking sessions documented in Chapter 4 reveal both the capabilities and current limitations of the system: stable, repeatable 3D position estimates during continuous LED visibility, and characteristic Z-spike artefacts during LED occlusion events that the system self-recovers from automatically.

The literature survey in Chapter 2 contextualises this work within the broader landscape of computer vision research. The foundational contributions of NeRF (Mildenhall et al., 2020), Vision Transformers (Dosovitskiy et al., 2021), YOLO-based multi-object tracking (Kader et al., 2025), and ORB-SLAM3 (Campos et al., 2021) collectively form the academic framework within which this project is situated.

7.2 Scope for Future Work

The following extensions are planned or recommended to enhance system capability:

- Multi-camera MediaPipe fusion: Triangulate MediaPipe 2D body landmark detections from all four calibrated Logitech cameras using DLT triangulation to produce metric 3D body poses — replacing the current single-camera relative-depth limitation with a calibrated multi-view approach.
- Colour LED tracking: Replace grayscale brightness thresholding with HSV colour-space dual-constraint detection for more robust LED detection under variable ambient lighting.
- Kalman filter temporal consistency: Integrate a Kalman filter for LED 2D centroid tracking in each camera, providing motion-model-based detection validation and rejection of spatially inconsistent bright-region detections.
- YOLO-based multi-person tracking: Extend the system to track multiple people simultaneously using YOLOv8 or YOLO11 detection combined with DeepSORT (Kader et al., 2025).
- NeRF-based 3D scene capture: Use the calibrated multi-camera array as input to a Neural Radiance Field pipeline (Mildenhall et al., 2020) to capture photorealistic volumetric 3D models.

- ViT-based pose estimation backbone: Replace MediaPipe BlazePose with a Vision Transformer-based backbone (Dosovitskiy et al., 2021) for improved landmark accuracy under occlusion.
- SLAM integration: Add ORB-SLAM3-style simultaneous localisation and mapping to enable camera pose recovery and persistent 3D mapping across multi-session captures.
- Edge deployment: Port the LED detection pipeline to an NVIDIA Jetson Orin or Raspberry Pi 5 for embedded standalone operation on the factory floor without requiring a laptop host.

References

- [1] Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). Simple online and realtime tracking. In 2016 IEEE International Conference on Image Processing (ICIP) (pp. 3464–3468). doi:10.1109/ICIP.2016.7533003.
- [2] Bradski, G. (2000). The OpenCV Library. Dr. Dobb's Journal of Software Tools. Intel Corporation.
- [3] Campos, C., Elvira, R., Gomez Rodriguez, J.J., Montiel, J.M.M., & Tardos, J.D. (2021). ORB-SLAM3: An Accurate Open-Source Library for Visual, Visual-Inertial and Multi-Map SLAM. IEEE Transactions on Robotics. doi:10.1109/TRO.2021.3075644. arXiv:2007.11898v2.
- [4] Cao, Z., Simon, T., Wei, S.E., & Sheikh, Y. (2017). Realtime multi-person 2D pose estimation using part affinity fields. In IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2017). IEEE.
- [5] Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Houlsby, N. (2021). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR 2021. arXiv:2010.11929v2.
- [6] Hartley, R., & Zisserman, A. (2003). Multiple View Geometry in Computer Vision (2nd ed.). Cambridge University Press. ISBN 9780521540513.
- [7] Intel Corporation. (2023). Intel RealSense SDK 2.0 Documentation. Available at: <https://dev.intelrealsense.com/>
- [8] Kader, T., Ab. Nasir, A.F., Toh, M.Z., Shapiee, M.N.A., & Abdul Razak, A.F. (2025). Human Detection and Tracking with YOLO and SORT Tracking Algorithm. International Journal of Advanced Computer Science and Applications (IJACSA), 16(5), 133–143.
- [9] Lugaresi, C., Tang, J., Nash, H., McClanahan, C., Uboweja, E., Hays, M., Zhang, F., Chang, C.L., Yong, M.G., Lee, J., Chang, W.T., Hua, W., Georg, M., & Grundmann, M. (2019). MediaPipe: A Framework for Building Perception Pipelines. arXiv:1906.08172.
- [10] Mildenhall, B., Srinivasan, P.P., Tancik, M., Barron, J.T., Ramamoorthi, R., & Ng, R. (2020). NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis. arXiv:2003.08934v2. ECCV 2020.
- [11] Wojke, N., Bewley, A., & Paulus, D. (2017). Simple online and realtime tracking with a deep association metric. In 2017 IEEE International Conference on Image Processing (ICIP) (pp. 3645–3649). doi:10.1109/ICIP.2017.8296962.
- [12] Zhang, Z. (2000). A Flexible New Technique for Camera Calibration. IEEE Transactions on Pattern Analysis and Machine Intelligence, 22(11), 1330–1334. doi:10.1109/34.888718.